100 Algorithms

in Python

Ordered by importance for coding interviews

Contents

How to Read This Book 1
Standard Helpers . . . 1

Topic Index 4

Tier 1 · Must Know Cold 6
1. Two Sum . . . 7
2. Prefix Sum + Hash Map . . . 8
3. Group Anagrams . . . 9
4. Reverse a Linked List . . . 10
5. 3Sum . . . 11
6. Floyd's Tortoise and Hare . . . 12
7. Longest Substring Without Repeating Characters . . . 14
8. Minimum Window Substring . . . 15
9. Binary Search . . . 16
10. Binary Search on the Answer . . . 18
11. Merge Sort . . . 19
12. Quick Sort . . . 20
13. Top-K Elements . . . 21
14. Merge K Sorted Arrays . . . 22
15. Find Median from Data Stream . . . 23
16. Valid Parentheses . . . 24
17. Monotonic Stack . . . 25
18. Merge Intervals . . . 26
19. Meeting Rooms II . . . 27
20. Kadane's Algorithm . . . 28
21. Best Time to Buy and Sell Stock . . . 29
22. Subsets . . . 30
23. Permutations . . . 31
24. Binary Tree Level-Order Traversal . . . 32
25. Diameter and Max Path Sum . . . 33
26. Lowest Common Ancestor . . . 35
27. Validate a Binary Search Tree . . . 36
28. Depth-First Search on a Graph . . . 37
29. Number of Islands . . . 39
30. BFS Shortest Path . . . 40
31. Kahn's Algorithm . . . 41
32. Union-Find . . . 42
33. Dijkstra's Algorithm . . . 44
34. Trie . . . 45
35. LRU Cache . . . 47

Tier 2 · Should Know Well **49**
36. Hash Map with Separate Chaining . . . 50
37. Trapping Rain Water . . . 52
38. Dutch National Flag . . . 53
39. Search in a Rotated Sorted Array . . . 54
40. Median of Two Sorted Arrays . . . 55
41. Heapify . . . 56
42. Quickselect . . . 58
43. Bucket Sort . . . 59
44. Largest Rectangle in Histogram . . . 60
45. Basic Calculator . . . 61
46. Sliding Window Maximum . . . 62
47. Jump Game . . . 63
48. Jump Game II . . . 64
49. Gas Station . . . 65
50. Interval Scheduling . . . 66
51. Task Scheduler . . . 67
52. Partition Labels . . . 68
53. Combination Sum . . . 69
54. Word Search . . . 70
55. Serialize and Deserialize a Binary Tree . . . 71
56. BST Search, Insert, Delete . . . 73
57. BST Iterator . . . 75
58. Multi-Source BFS . . . 76
59. Word Ladder . . . 77
60. Cycle Detection in a Directed Graph . . . 78
61. Topological Sort with DFS . . . 79
62. Connected Components . . . 80
63. Word Search II . . . 81
64. KMP String Matching . . . 83
65. Longest Palindromic Substring . . . 84
66. Encode and Decode Strings . . . 85
67. Fibonacci . . . 86
68. Single Number . . . 88
69. Insert Delete GetRandom O(1) . . . 89
70. Time-Based Key-Value Store . . . 90

Tier 3 · Good to Know **91**
71. Hash Map with Open Addressing . . . 92
72. Boyer-Moore Majority Vote . . . 94
73. Median of K Sorted Arrays . . . 95
74. Counting Sort . . . 97
75. Heap Sort . . . 98
76. Circular Queue . . . 99
77. Candy . . . 101
78. Hand of Straights . . . 102
79. N-Queens . . . 103
80. Bipartite Check . . . 104
81. Cycle Detection in an Undirected Graph . . . 105

82. Bellman-Ford . . . 106
83. Kruskal's Minimum Spanning Tree . . . 107
84. Prim's Minimum Spanning Tree . . . 109
85. Rabin-Karp . . . 110
86. Longest Common Prefix . . . 111
87. Text Justification . . . 112
88. String Compression . . . 114
89. Reverse Words in a String . . . 115
90. Multiply Strings . . . 116
91. GCD and LCM . . . 117
92. Sieve of Eratosthenes . . . 118
93. Fast Exponentiation . . . 119
94. Reservoir Sampling . . . 120
95. Orientation and Segment Intersection . . . 121
96. Convex Hull . . . 122
97. Max Points on a Line . . . 123
98. Count Set Bits . . . 124
99. Bitmask Subset Enumeration . . . 125
100. Rate Limiter . . . 126

How to Read This Book

This book holds the same one hundred algorithms as *100 Algorithms in Pseudocode*, in the same order, as Python you can run. The order is importance for a coding interview. Tier 1, the first thirty-five, are the algorithms to know cold; Tier 2, the next thirty-five, are the ones to know well; Tier 3, the last thirty, are good to know. Inside a tier, related algorithms sit next to each other.

Each entry states the problem with a tiny example and then shows one complete file. A file has up to three parts: the helpers, when the input is a tree or a linked list; the algorithm, where every function opens with a one-line docstring that gives the idea and its cost; and a try-it section of prints whose expected output follows `# ->` on the same line. Every file runs on its own, for example `python3 algorithms_100/python/001_two_sum.py`, and every printed line was checked against its comment. The variable names match the pseudocode book line for line, so the two books can be read side by side.

The Python is simple on purpose. Loops are written out instead of comprehensions, there are no lambdas or one-line tricks, and names say what they hold. The aim is that you read a function once, close the book and write it again. The toolbox is small.

```
from collections import deque    # a queue: append() and popleft() are O(1)
import heapq                     # a min-heap on a list: heappush, heappop, heap[0]
import math                      # math.inf, math.sqrt, math.ceil
import random                    # random pivots and sampling
import bisect                    # ready-made binary search (search chapter only)

# sorted(items) returns a new sorted list; items.sort() sorts in place.
# Both are O(n log n) and stable, and tuples sort field by field.
```

Standard Helpers

Entries whose input is a binary tree or a linked list build it from a plain Python list, so a test reads like the LeetCode example. `build_tree` takes a level-order list with `None` for a missing child, and `tree_to_list` turns a tree back into one; `build_list` and `to_list` do the same for linked lists. Every file carries its own copy so it runs on its own; the book prints them once, here, and each entry names the ones it uses.

```
from collections import deque      # popleft is O(1)


class TreeNode:
    def __init__(self, val, left=None, right=None):
        self.val = val
        self.left = left
        self.right = right
```

```
def build_tree(values):
    """Build a tree from a level-order list with None for missing children."""
    if not values or values[0] is None:
        return None
    root = TreeNode(values[0])
    queue = deque([root])
    i = 1
    while queue and i < len(values):
        node = queue.popleft()
        if i < len(values) and values[i] is not None:
            node.left = TreeNode(values[i])
            queue.append(node.left)
        i += 1
        if i < len(values) and values[i] is not None:
            node.right = TreeNode(values[i])
            queue.append(node.right)
        i += 1
    return root


def tree_to_list(root):
    """Level-order list with None for gaps, trailing Nones removed."""
    values = []
    queue = deque([root])
    while queue:
        node = queue.popleft()
        if node is None:
            values.append(None)
            continue
        values.append(node.val)
        queue.append(node.left)
        queue.append(node.right)
    while values and values[-1] is None:
        values.pop()
    return values


class ListNode:
    def __init__(self, val, next=None):
        self.val = val
        self.next = next


def build_list(values):
    """Build a linked list from a Python list and return its head."""
    dummy = ListNode(0)
    tail = dummy
    for value in values:
        tail.next = ListNode(value)
        tail = tail.next
    return dummy.next


def to_list(head):
```

```
    """Read a linked list back into a Python list."""
    values = []
    while head is not None:
        values.append(head.val)
        head = head.next
    return values
```

Topic Index

The book is ordered by importance. This index groups the same entries by topic; every number is a link.

Topic	Entries
Arrays & Hashing	#1 Two Sum · #2 Prefix Sum + Hash Map · #72 Boyer-Moore Majority Vote
Hash Map Internals	#36 Hash Map with Separate Chaining · #71 Hash Map with Open Addressing
Two Pointers	#4 Reverse a Linked List · #5 3Sum · #37 Trapping Rain Water · #38 Dutch National Flag
Sliding Window	#7 Longest Substring Without Repeating Characters · #8 Minimum Window Substring
Binary Search	#9 Binary Search · #10 Binary Search on the Answer · #39 Search in a Rotated Sorted Array · #40 Median of Two Sorted Arrays · #73 Median of K Sorted Arrays
Sorting	#11 Merge Sort · #12 Quick Sort · #43 Bucket Sort · #74 Counting Sort · #75 Heap Sort
Heaps	#13 Top-K Elements · #14 Merge K Sorted Arrays · #41 Heapify · #42 Quickselect
Stacks	#16 Valid Parentheses · #17 Monotonic Stack · #44 Largest Rectangle in Histogram · #45 Basic Calculator
Queues	#46 Sliding Window Maximum · #76 Circular Queue
Intervals	#18 Merge Intervals · #19 Meeting Rooms II
Greedy	#20 Kadane's Algorithm · #21 Best Time to Buy and Sell Stock · #47 Jump Game · #48 Jump Game II · #49 Gas Station · #50 Interval Scheduling · #51 Task Scheduler · #52 Partition Labels · #77 Candy · #78 Hand of Straights
Backtracking	#22 Subsets · #23 Permutations · #53 Combination Sum · #54 Word Search · #79 N-Queens
DFS	#25 Diameter and Max Path Sum · #26 Lowest Common Ancestor · #28 Depth-First Search on a Graph · #29 Number of Islands · #55 Serialize and Deserialize a Binary Tree
BFS	#24 Binary Tree Level-Order Traversal · #30 BFS Shortest Path · #58 Multi-Source BFS · #59 Word Ladder · #80 Bipartite Check
Cycle Detection	#6 Floyd's Tortoise and Hare · #60 Cycle Detection in a Directed Graph · #81 Cycle Detection in an Undirected Graph
Topological Sort	#31 Kahn's Algorithm · #61 Topological Sort with DFS
Connected Components	#32 Union-Find · #62 Connected Components
Shortest Path	#33 Dijkstra's Algorithm · #82 Bellman-Ford
MST	#83 Kruskal's Minimum Spanning Tree · #84 Prim's Minimum Spanning Tree
BST	#27 Validate a Binary Search Tree · #56 BST Search, Insert, Delete · #57 BST Iterator

Topic	Entries
Tries	#34 Trie · #63 Word Search II
Strings	#3 Group Anagrams · #64 KMP String Matching · #65 Longest Palindromic Substring · #66 Encode and Decode Strings · #85 Rabin-Karp · #86 Longest Common Prefix · #87 Text Justification · #88 String Compression · #89 Reverse Words in a String · #90 Multiply Strings
Math	#67 Fibonacci · #91 GCD and LCM · #92 Sieve of Eratosthenes · #93 Fast Exponentiation · #94 Reservoir Sampling
Geometry	#95 Orientation and Segment Intersection · #96 Convex Hull · #97 Max Points on a Line
Bit Manipulation	#68 Single Number · #98 Count Set Bits · #99 Bitmask Subset Enumeration
Design	#15 Find Median from Data Stream · #35 LRU Cache · #69 Insert Delete GetRandom O(1) · #70 Time-Based Key-Value Store · #100 Rate Limiter

Tier 1 · Must Know Cold

These thirty-five algorithms are the core of the interview. Each one is the standard answer to a whole family of problems: hashing to remember the past, two pointers and sliding windows for sequences, binary search for anything monotone, sorting and heaps for order, stacks for nesting, BFS and DFS for every tree and graph, Kahn's algorithm, union-find and Dijkstra for dependencies, components and distances, the trie for prefixes and the LRU cache for design. Learn them until you can write each one from a blank page without hesitation.

1. Two Sum

Hash Map of Complements · Arrays & Hashing · Tier 1 · python/001_two_sum.py

Given an array of integers and a target, return the indices of the two numbers that add up to the target (exactly one pair exists): [2, 7, 11, 15], target 9 → [0, 1].

```python
# --- algorithm ---
def two_sum(nums, target):
    """One pass; look up target - x before storing x. O(n) time and space."""
    seen = {}                       # value -> index
    for i in range(len(nums)):
        need = target - nums[i]
        if need in seen:            # partner appeared earlier
            return [seen[need], i]
        seen[nums[i]] = i           # store only after checking
    return []


# --- try it ---
print(two_sum([2, 7, 11, 15], 9))   # -> [0, 1]
print(two_sum([3, 2, 4], 6))        # -> [1, 2]
print(two_sum([3, 3], 6))           # -> [0, 1]
print(two_sum([1, 2], 7))           # -> []
```

2. Prefix Sum + Hash Map

Subarray Sum Equals K · Arrays & Hashing · Tier 1 · python/002_prefix_sum_hash_map.py

Given an array of integers (negatives allowed) and an integer k, count the contiguous subarrays whose sum is exactly k: [2, -1, 1, 2], k 2 → 4.

```python
# --- algorithm ---
def subarray_sum(nums, k):
    """Count earlier prefix sums equal to prefix - k. O(n) time and space."""
    prefix_count = {}               # prefix sum -> how many times seen
    prefix_count[0] = 1             # the empty prefix, before index 0
    prefix = 0
    total = 0
    for x in nums:
        prefix += x                 # sum of nums up to and including x
        # each earlier prefix equal to prefix - k starts one subarray of sum k
        total += prefix_count.get(prefix - k, 0)
        prefix_count[prefix] = prefix_count.get(prefix, 0) + 1   # record last
    return total


# --- try it ---
print(subarray_sum([2, -1, 1, 2], 2))   # -> 4
print(subarray_sum([1, 1, 1], 2))       # -> 2
print(subarray_sum([1, 2, 3], 3))       # -> 2
print(subarray_sum([0, 0, 0], 0))       # -> 6
```

3. Group Anagrams

Canonical-Key Hashing · Strings · Tier 1 · python/003_group_anagrams.py

Given a list of lowercase words, group the words that are anagrams of each other (same letters, same counts), groups in first-seen order: [“eat”, “tea”, “tan”, “ate”, “nat”, “bat”] → [[“eat”, “tea”, “ate”], [“tan”, “nat”], [“bat”]].

```python
# --- algorithm ---
def letter_counts(word):
    """The 26 letter counts of a word, as a tuple: anagrams share it. O(L)."""
    counts = [0] * 26
    for ch in word:
        slot = ord(ch) - ord('a')   # 'a' -> 0, 'b' -> 1, ..., 'z' -> 25
        counts[slot] += 1
    return tuple(counts)            # a tuple can be a dict key; a list cannot


def group_anagrams(words):
    """Bucket every word under its count key. O(n * L) time and space."""
    groups = {}                     # count key -> words with that key
    for word in words:
        key = letter_counts(word)
        if key not in groups:
            groups[key] = []        # first word of a new group
        groups[key].append(word)
    return list(groups.values())    # one list per key, in first-seen order


# --- try it ---
groups = group_anagrams(["eat", "tea", "tan", "ate", "nat", "bat"])
print(groups)   # -> [['eat', 'tea', 'ate'], ['tan', 'nat'], ['bat']]
print(group_anagrams(["", "b", ""]))    # -> [['', ''], ['b']]
print(group_anagrams(["aab", "abb"]))   # -> [['aab'], ['abb']]
print(group_anagrams(["ad", "bc"]))     # -> [['ad'], ['bc']]
```

4. Reverse a Linked List

Three Pointers: prev, curr, next · Two Pointers · Tier 1 · python/004_reverse_linked_list.py

Given the head of a singly linked list, reverse the list in place and return the new head. As values: [1, 2, 3, 4, 5] → [5, 4, 3, 2, 1].

```python
# --- helpers ---
# standard helpers ListNode, build_list, to_list: printed once under Standard Helpers


# --- algorithm ---
def reverse_list(head):
    """Turn every next arrow around in one walk. O(n) time, O(1) space."""
    prev = None                     # head of the reversed part
    curr = head                     # head of the part still to reverse
    while curr is not None:
        next_node = curr.next       # save the rest before the cut
        curr.next = prev            # turn this arrow around
        prev = curr                 # the reversed part gains curr
        curr = next_node            # step into the saved rest
    return prev                     # curr ran off the end: prev is the new head


# --- try it ---
head = build_list([1, 2, 3, 4, 5])
print(to_list(reverse_list(head)))                  # -> [5, 4, 3, 2, 1]
print(to_list(reverse_list(build_list([1, 2]))))    # -> [2, 1]
print(to_list(reverse_list(build_list([7]))))       # -> [7]
print(to_list(reverse_list(None)))                  # -> []
```

5. 3Sum

Sort, Fix One, Converge Two Pointers · Two Pointers · Tier 1 · python/005_three_sum.py

Given an array of integers, return every triplet of values that sums to zero, each distinct triplet once: [-1, 0, 1, 2, -1, -4] → [[-1, -1, 2], [-1, 0, 1]].

```python
# --- algorithm ---
def three_sum(nums):
    """Sort; fix nums[i], two pointers find pairs summing to -nums[i]. O(n²)."""
    nums = sorted(nums)             # sorted copy: equal values sit together
    triplets = []
    n = len(nums)
    for i in range(n - 2):
        if nums[i] > 0:
            break                   # smallest of the three is positive: done
        if i > 0 and nums[i] == nums[i - 1]:
            continue                # same first value, same triplets
        left = i + 1
        right = n - 1
        while left < right:         # Two Sum II on nums[i+1..n-1]
            total = nums[i] + nums[left] + nums[right]
            if total < 0:
                left += 1           # too small: a bigger second value
            elif total > 0:
                right -= 1          # too big: a smaller third value
            else:
                triplets.append([nums[i], nums[left], nums[right]])
                left += 1
                right -= 1
                while left < right and nums[left] == nums[left - 1]:
                    left += 1       # skip repeats of the second value
    return triplets


# --- try it ---
print(three_sum([-1, 0, 1, 2, -1, -4]))     # -> [[-1, -1, 2], [-1, 0, 1]]
print(three_sum([0, 0, 0, 0]))              # -> [[0, 0, 0]]
print(three_sum([0, 1, 1]))                 # -> []
print(three_sum([-2, 0, 0, 2, 2]))          # -> [[-2, 0, 2]]
```

6. Floyd's Tortoise and Hare

Detect a Cycle and Find Its Entry · Cycle Detection · Tier 1 · python/006_floyd_cycle_detection.py

Given the head of a linked list whose tail may link back to an earlier node, return the node where the cycle begins (None if there is no cycle) in O(1) extra space: values 1..7, tail linked to index 2 → the node with value 3.

```python
# --- helpers ---
# standard helpers ListNode: printed once under Standard Helpers


def build_cycle_list(values, pos):
    """Build a list whose tail links back to index pos (-1: no cycle)."""
    if len(values) == 0:
        return None
    nodes = []
    for value in values:
        nodes.append(ListNode(value))
    for i in range(len(nodes) - 1):
        nodes[i].next = nodes[i + 1]
    if pos >= 0:
        nodes[-1].next = nodes[pos]     # close the loop
    return nodes[0]


# --- algorithm ---
def meeting_point(head):
    """Slow steps 1, fast steps 2; they meet only inside a cycle. O(n)."""
    slow = head
    fast = head
    while fast is not None and fast.next is not None:
        slow = slow.next
        fast = fast.next.next
        if slow is fast:            # the same node, not just an equal value
            return slow
    return None                     # fast fell off the end: no cycle


def cycle_entry(head):
    """From head and from the meeting point, 1 step each: they meet at the
    entry. O(n) time, O(1) space."""
    meet = meeting_point(head)
    if meet is None:
        return None
    from_head = head
    from_meet = meet
    while from_head is not from_meet:   # both are a steps from the entry
        from_head = from_head.next
        from_meet = from_meet.next
    return from_head


# --- try it ---
head = build_cycle_list([1, 2, 3, 4, 5, 6, 7], 2)
print(meeting_point(head).val)                          # -> 6
```

```
print(cycle_entry(head).val)                              # -> 3
print(cycle_entry(build_cycle_list([3, 2, 0, -4], 1)).val)  # -> 2
print(cycle_entry(build_cycle_list([1, 2, 3], -1)))     # -> None
```

7. Longest Substring Without Repeating Characters

Variable Window with Last-Seen Index · Sliding Window · Tier 1 · python/007_longest_substring_without_repeating.py

Given a string, return the length of its longest substring in which no character appears twice: “abcbad” → 4 (the window “cbad”).

```python
# --- algorithm ---
def length_of_longest_substring(s):
    """Window s[left..right] has no repeats; on a repeat inside it, jump left
    past the earlier copy. O(n) time, O(alphabet) space."""
    last_seen = {}                  # char -> index of its latest occurrence
    left = 0
    best = 0
    for right in range(len(s)):
        ch = s[right]
        if ch in last_seen and last_seen[ch] >= left:
            left = last_seen[ch] + 1    # jump past the copy in the window
        last_seen[ch] = right
        best = max(best, right - left + 1)
    return best


# --- try it ---
print(length_of_longest_substring("abcbad"))      # -> 4
print(length_of_longest_substring("abcabcbb"))    # -> 3
print(length_of_longest_substring("abba"))        # -> 2
print(length_of_longest_substring("bbbbb"))       # -> 1
print(length_of_longest_substring(""))            # -> 0
```

8. Minimum Window Substring

Expand Right, Shrink Left · Sliding Window · Tier 1 · python/008_minimum_window_substring.py

Given strings s and t, return the shortest substring of s that contains every character of t, repeats included (“” if there is none): s “ADOBECODEBANC”, t “ABC” → “BANC”.

```python
# --- algorithm ---
def char_counts(text):
    """How many copies of each character text holds. O(len(text))."""
    counts = {}
    for ch in text:
        counts[ch] = counts.get(ch, 0) + 1
    return counts


def min_window(s, t):
    """Grow right to cover t, shrink left while covered. O(len(s) + len(t))."""
    if len(t) == 0:
        return ""                   # nothing to cover
    need = char_counts(t)           # char -> copies the window lacks
    missing = len(t)                # chars of t not yet covered, with repeats
    left = 0
    best_start = 0
    best_len = len(s) + 1           # longer than any real window
    for right in range(len(s)):
        ch = s[right]
        if need.get(ch, 0) > 0:
            missing -= 1            # this copy fills a real gap
        need[ch] = need.get(ch, 0) - 1  # below zero: a spare copy
        while missing == 0:         # covers t: record, then shrink
            if right - left + 1 < best_len:
                best_start = left
                best_len = right - left + 1
            drop = s[left]
            need[drop] += 1
            if need[drop] > 0:
                missing += 1        # dropped a copy the window needed
            left += 1
    if best_len > len(s):
        return ""                   # no window ever covered t
    return s[best_start:best_start + best_len]


# --- try it ---
print(min_window("ADOBECODEBANC", "ABC"))       # -> BANC
print(min_window("AAB", "AA"))                  # -> AA
print(min_window("a", "a"))                     # -> a
print(min_window("a", "aa") == "")              # -> True
```

9. Binary Search

The Lower-Bound Template · Binary Search · Tier 1 · python/009_binary_search_lower_bound.py

Given a sorted array and a target, return the first index whose value is at least the target, or len(nums) if every value is smaller: [1, 3, 3, 5, 8, 9], target 3 → 1.

```python
# --- algorithm ---
def lower_bound(nums, target):
    """First index i with nums[i] >= target, len(nums) if none. O(log n)."""
    lo = 0
    hi = len(nums)              # half-open [lo, hi); len(nums) means "none"
    while lo < hi:              # the window still holds an unknown index
        mid = (lo + hi) // 2
        if nums[mid] < target:
            lo = mid + 1        # mid and everything left of it are too small
        else:
            hi = mid            # mid qualifies: the answer is mid or left of it
    return lo


def upper_bound(nums, target):
    """First i with nums[i] > target: the same loop, <= for <. O(log n)."""
    lo = 0
    hi = len(nums)
    while lo < hi:
        mid = (lo + hi) // 2
        if nums[mid] <= target:     # equal values count as too small here
            lo = mid + 1
        else:
            hi = mid
    return lo


def search(nums, target):
    """Exact search: a present target sits at its lower bound. O(log n)."""
    i = lower_bound(nums, target)
    if i < len(nums) and nums[i] == target:
        return i
    return -1


def search_range(nums, target):
    """First and last index, [-1, -1] if absent: two searches, O(log n)."""
    first = lower_bound(nums, target)
    if first == len(nums) or nums[first] != target:
        return [-1, -1]
    return [first, upper_bound(nums, target) - 1]


# --- try it ---
nums = [1, 3, 3, 5, 8, 9]
print(lower_bound(nums, 3))         # -> 1
print(lower_bound(nums, 4))         # -> 3
print(search(nums, 4))              # -> -1
print(search_range(nums, 3))        # -> [1, 2]
```

```
print(lower_bound(nums, 10))        # -> 6
```

10. Binary Search on the Answer

Search a Monotone Yes/No Predicate · Binary Search · Tier 1 · python/010_binary_search_on_answer.py

Koko eats k bananas per hour from a single pile; when fewer than k are left in that pile she still spends the whole hour on it. Return the smallest k that finishes every pile within h hours (h is at least the number of piles): piles [3, 6, 7, 11], h 8 → 4.

```python
import math                          # math.ceil rounds a division up


# --- algorithm ---
def can_finish(piles, h, speed):
    """The yes/no check: are all piles eaten within h hours? O(n)."""
    hours = 0
    for pile in piles:
        hours += math.ceil(pile / speed)    # a partial hour costs a full hour
    return hours <= h


def min_eating_speed(piles, h):
    """First speed in [1, max(piles)] where can_finish says yes.
    O(n log m) for m = max(piles)."""
    lo = 1                           # the slowest possible speed
    hi = max(piles)                  # one pile per hour: always fast enough
    while lo < hi:
        mid = (lo + hi) // 2
        if can_finish(piles, h, mid):
            hi = mid                 # mid works; a slower speed might too
        else:
            lo = mid + 1             # mid fails, and so does every speed below
    return lo


# --- try it ---
print(min_eating_speed([3, 6, 7, 11], 8))           # -> 4
print(min_eating_speed([30, 11, 23, 4, 20], 5))     # -> 30
print(min_eating_speed([30, 11, 23, 4, 20], 6))     # -> 23
print(can_finish([3, 6, 7, 11], 8, 3))              # -> False
```

11. Merge Sort

Divide, Sort Halves, Merge · Sorting · Tier 1 · python/011_merge_sort.py

Sort an array of integers in ascending order and return the result as a new list: [5, 2, 4, 7, 1, 3, 2, 6] → [1, 2, 2, 3, 4, 5, 6, 7].

```python
# --- algorithm ---
def merge(left, right):
    """Two sorted lists into one: keep taking the smaller front. O(n) time."""
    merged = []
    i = 0                               # front of left
    j = 0                               # front of right
    while i < len(left) and j < len(right):
        if left[i] <= right[j]:         # a tie takes the left one: stable
            merged.append(left[i])
            i += 1
        else:
            merged.append(right[j])
            j += 1
    while i < len(left):                # one side is used up: copy the rest
        merged.append(left[i])
        i += 1
    while j < len(right):
        merged.append(right[j])
        j += 1
    return merged


def merge_sort(nums):
    """Sort each half recursively, then merge. O(n log n) time, O(n) space."""
    if len(nums) <= 1:
        return list(nums)               # 0 or 1 values: sorted already (copy)
    mid = len(nums) // 2
    left = merge_sort(nums[:mid])       # nums[0..mid-1]
    right = merge_sort(nums[mid:])      # nums[mid..n-1]
    return merge(left, right)


# --- try it ---
print(merge_sort([5, 2, 4, 7, 1, 3, 2, 6]))   # -> [1, 2, 2, 3, 4, 5, 6, 7]
print(merge_sort([3, -1, 0, -1]))             # -> [-1, -1, 0, 3]
print(merge([1, 4], [2, 3]))                  # -> [1, 2, 3, 4]
print(merge_sort([]))                         # -> []
```

12. Quick Sort

Partition Around a Random Pivot · Sorting · Tier 1 · python/012_quick_sort.py

Sort an array of integers in place, without a second array: [6, 3, 7, 4, 1, 5, 2] → [1, 2, 3, 4, 5, 6, 7].

```python
import random   # random.randint(a, b) can return a or b too


# --- algorithm ---
def partition(nums, lo, hi):
    """Lomuto partition of nums[lo..hi] around a random pivot; return the
    pivot's final index. O(hi - lo) time."""
    pick = random.randint(lo, hi)
    nums[pick], nums[hi] = nums[hi], nums[pick]  # park the pivot at the end
    pivot = nums[hi]
    store = lo                       # invariant: nums[lo..store-1] < pivot
    for i in range(lo, hi):
        if nums[i] < pivot:          # a small value joins the left region
            nums[store], nums[i] = nums[i], nums[store]
            store += 1
    nums[store], nums[hi] = nums[hi], nums[store]  # pivot between the regions
    return store


def sort_range(nums, lo, hi):
    """Partition nums[lo..hi], then sort each side. Expected O(n log n) on
    distinct values; many equal values make it O(n^2), n levels deep."""
    if lo >= hi:                     # 0 or 1 values: sorted already
        return
    p = partition(nums, lo, hi)
    sort_range(nums, lo, p - 1)      # the pivot at p is final: skip it
    sort_range(nums, p + 1, hi)


def quick_sort(nums):
    """Sort the whole array in place and return it. Expected O(n log n)."""
    sort_range(nums, 0, len(nums) - 1)
    return nums


# --- try it ---
print(quick_sort([6, 3, 7, 4, 1, 5, 2]))   # -> [1, 2, 3, 4, 5, 6, 7]
print(quick_sort([2, -3, 2, 0, -3, 2]))    # -> [-3, -3, 0, 2, 2, 2]
print(quick_sort([1]))                     # -> [1]
print(quick_sort([]))                      # -> []
```

13. Top-K Elements

Keep a Size-K Min-Heap · Heaps · Tier 1 · python/013_top_k_heap.py

Given an unsorted array and k, return the kth largest value (duplicates count); the same heap also holds the k largest values themselves: [3, 1, 5, 12, 2, 11], k = 3 → 5 (the three largest are 12, 11 and 5).

```python
import heapq   # heappush / heappop keep the smallest at index 0


# --- algorithm ---
def top_k(nums, k):
    """Min-heap of the k largest values so far. O(n log k) time, O(k) space."""
    heap = []
    for value in nums:
        heapq.heappush(heap, value)
        if len(heap) > k:               # one too many: drop the smallest
            heapq.heappop(heap)
    return heap


def kth_largest(nums, k):
    """The top of the size-k min-heap is the kth largest. O(n log k)."""
    heap = top_k(nums, k)
    return heap[0]                      # the smallest of the k largest


# --- try it ---
print(kth_largest([3, 1, 5, 12, 2, 11], 3))          # -> 5
print(sorted(top_k([3, 1, 5, 12, 2, 11], 3)))        # -> [5, 11, 12]
print(kth_largest([3, 2, 3, 1, 2, 4, 5, 5, 6], 4))   # -> 4
print(kth_largest([-1, -1], 2))                      # -> -1
```

14. Merge K Sorted Arrays

A Heap of the Front Elements · Heaps · Tier 1 · python/014_merge_k_sorted.py

Merge k sorted arrays of integers into one sorted array: [[1, 4, 7], [2, 5], [0, 3, 6]] → [0, 1, 2, 3, 4, 5, 6, 7].

```python
import heapq   # heappush / heappop keep the smallest at index 0


# --- algorithm ---
def merge_k_sorted(arrays):
    """Heap of each array's front as (value, row, col); pop the smallest and
    push its successor. O(N log k) time for N values in k arrays."""
    heap = []                                   # of (value, row, col)
    for row in range(len(arrays)):
        if len(arrays[row]) > 0:                # an empty array has no front
            heapq.heappush(heap, (arrays[row][0], row, 0))
    merged = []
    while len(heap) > 0:
        value, row, col = heapq.heappop(heap)   # the smallest front of all
        merged.append(value)
        if col + 1 < len(arrays[row]):          # that row has a next value
            heapq.heappush(heap, (arrays[row][col + 1], row, col + 1))
    return merged


# --- try it ---
arrays = [[1, 4, 7], [2, 5], [0, 3, 6]]
print(merge_k_sorted(arrays))                   # -> [0, 1, 2, 3, 4, 5, 6, 7]
print(merge_k_sorted([[2, 2], [], [-1, 2]]))   # -> [-1, 2, 2, 2]
print(merge_k_sorted([[5]]))                    # -> [5]
print(merge_k_sorted([]))                       # -> []
```

15. Find Median from Data Stream

Two Heaps Meet in the Middle · Design · Tier 1 · python/015_median_from_data_stream.py

Numbers arrive one at a time; after any arrival, report the median of all the numbers so far (the mean of the two middle values when the count is even): add 5, 15, 1, 3 → medians 5, 10.0, 5, 4.0.

```python
import heapq            # heappush / heappop keep the smallest at index 0


# --- algorithm ---
class MedianFinder:
    """Max-heap of the smaller half, min-heap of the larger half."""

    def __init__(self):
        self.low = []           # max-heap of the smaller half, stored negated
        self.high = []          # min-heap of the larger half

    def add_num(self, num):
        """Pass num through low into high, then rebalance. O(log n)."""
        heapq.heappush(self.low, -num)
        top_low = -heapq.heappop(self.low)    # the largest of low and num
        heapq.heappush(self.high, top_low)
        if len(self.high) > len(self.low):    # low may lead by one, not trail
            top_high = heapq.heappop(self.high)
            heapq.heappush(self.low, -top_high)

    def find_median(self):
        """Read the middle from the two tops. O(1)."""
        if len(self.low) > len(self.high):
            return -self.low[0]               # odd count: low holds the middle
        return (-self.low[0] + self.high[0]) / 2


# --- try it ---
finder = MedianFinder()
finder.add_num(5)
print(finder.find_median())     # -> 5
finder.add_num(15)
print(finder.find_median())     # -> 10.0
finder.add_num(1)
print(finder.find_median())     # -> 5
finder.add_num(3)
print(finder.find_median())     # -> 4.0
```

16. Valid Parentheses

The Matching Stack · Stacks · Tier 1 · python/016_valid_parentheses.py

Given a string of the brackets ()[]{}, decide whether every opener is closed by a bracket of the same kind, innermost first: “([]{})” → True, “([)]” → False.

```python
# --- algorithm ---
def is_valid(s):
    """Push openers; every closer must match the top. O(n) time and space."""
    partner = {")": "(", "]": "[", "}": "{"}     # closer -> its opener
    stack = []                                   # openers still waiting
    for ch in s:
        if ch in partner:
            if len(stack) == 0 or stack[-1] != partner[ch]:
                return False                     # nothing open, or wrong kind
            stack.pop()                          # the innermost opener closes
        else:
            stack.append(ch)
    return len(stack) == 0                       # leftovers never closed


# --- try it ---
print(is_valid("([]{})"))       # -> True
print(is_valid("([)]"))         # -> False
print(is_valid("(("))           # -> False
print(is_valid(")("))           # -> False
print(is_valid(""))             # -> True
```

17. Monotonic Stack

Next Greater Element · Stacks · Tier 1 · python/017_monotonic_stack.py

For each day's temperature, return how many days you must wait for a warmer one, or 0 if no warmer day comes: [73, 74, 75, 71, 69, 72, 76, 73] → [1, 1, 4, 2, 1, 1, 0, 0].

```python
# --- algorithm ---
def daily_temperatures(temperatures):
    """Stack of days still waiting; a warmer day answers them. O(n) time."""
    n = len(temperatures)
    answer = [0] * n                # 0 stays for days nothing beats
    stack = []                      # waiting indices, temps never rising
    for i in range(n):
        while len(stack) > 0 and temperatures[stack[-1]] < temperatures[i]:
            j = stack.pop()         # day i is the first warmer day after j
            answer[j] = i - j
        stack.append(i)             # day i waits for its own answer
    return answer


# --- try it ---
temperatures = [73, 74, 75, 71, 69, 72, 76, 73]
print(daily_temperatures(temperatures))     # -> [1, 1, 4, 2, 1, 1, 0, 0]
print(daily_temperatures([30, 40, 50]))     # -> [1, 1, 0]
print(daily_temperatures([70, 70, 71]))     # -> [2, 1, 0]
print(daily_temperatures([]))               # -> []
```

18. Merge Intervals

Sort by Start, Sweep and Extend · Intervals · Tier 1 · python/018_merge_intervals.py

Given closed intervals [start, end] in any order, merge every group that overlaps or touches and return the result sorted by start: [[1, 3], [8, 12], [2, 6], [9, 10], [12, 14]] → [[1, 6], [8, 14]].

```python
# --- algorithm ---
def merge_intervals(intervals):
    """Sort by start; stretch the last merged interval or open a new one.
    O(n log n) time for the sort, O(n) space for the output."""
    merged = []
    for start, end in sorted(intervals):    # pairs sort by start first
        if len(merged) > 0 and start <= merged[-1][1]:
            merged[-1][1] = max(merged[-1][1], end)   # overlap: stretch it
        else:
            merged.append([start, end])     # a gap: a new interval begins
    return merged


# --- try it ---
intervals = [[1, 3], [8, 12], [2, 6], [9, 10], [12, 14]]
print(merge_intervals(intervals))           # -> [[1, 6], [8, 14]]
print(merge_intervals([[1, 4], [4, 5]]))    # -> [[1, 5]]
print(merge_intervals([[1, 4], [5, 6]]))    # -> [[1, 4], [5, 6]]
print(merge_intervals([]))                  # -> []
```

19. Meeting Rooms II

Minimum Rooms with a Heap of End Times · Intervals · Tier 1 · python/019_meeting_rooms_ii.py

Given meetings as half-open intervals [start, end), return the fewest rooms that can host them all; a meeting may start the moment another one ends: [[0, 30], [5, 10], [10, 20], [15, 25]] → 3.

```python
import heapq            # heappush / heappop keep the smallest at index 0


# --- algorithm ---
def min_meeting_rooms(intervals):
    """Sort by start; a min-heap holds the time each room frees up.
    O(n log n) time, O(n) space."""
    ends = []                               # min-heap: one end time per room
    for start, end in sorted(intervals):    # earliest start first
        if len(ends) > 0 and ends[0] <= start:
            heapq.heappop(ends)             # the room that frees first is free
        heapq.heappush(ends, end)           # this room is busy until end
    return len(ends)


# --- try it ---
meetings = [[0, 30], [5, 10], [10, 20], [15, 25]]
print(min_meeting_rooms(meetings))                      # -> 3
print(min_meeting_rooms([[0, 30], [5, 10], [15, 20]]))  # -> 2
print(min_meeting_rooms([[7, 10], [2, 4]]))             # -> 1
print(min_meeting_rooms([]))                            # -> 0
```

20. Kadane's Algorithm

Maximum Subarray: Extend or Restart · Greedy · Tier 1 · python/020_kadane.py

Given a non-empty array of integers, return the largest sum of a contiguous subarray together with its first and last index: [-2, 1, -3, 4, -1, 2, 1, -5, 4] → (6, 3, 6), the subarray [4, -1, 2, 1].

```python
# --- algorithm ---
def max_subarray(nums):
    """Extend the run ending here, or restart at x. O(n) time, O(1) space."""
    ending_here = nums[0]           # best sum of a subarray that ends at i
    start = 0                       # where that subarray begins
    best = nums[0]                  # not 0: all may be negative
    best_start = 0
    best_end = 0
    for i in range(1, len(nums)):
        if ending_here < 0:         # a negative run only drags x down: restart
            ending_here = nums[i]
            start = i
        else:
            ending_here = ending_here + nums[i]     # extend the run
        if ending_here > best:      # improved: copy the bounds
            best = ending_here
            best_start = start
            best_end = i
    return best, best_start, best_end


# --- try it ---
print(max_subarray([-2, 1, -3, 4, -1, 2, 1, -5, 4]))   # -> (6, 3, 6)
print(max_subarray([5, 4, -1, 7, 8]))                  # -> (23, 0, 4)
print(max_subarray([-3, -1, -2]))                      # -> (-1, 1, 1)
print(max_subarray([7]))                               # -> (7, 0, 0)
```

21. Best Time to Buy and Sell Stock

Track the Running Minimum · Greedy · Tier 1 · python/021_best_time_stock.py

Given a stock's price on each day, buy on one day and sell on a later day to make the largest profit, or return 0 if no trade makes money: [7, 1, 5, 3, 6, 4] → 5 (buy at 1, sell at 6).

```python
import math                          # math.inf is higher than any price


# --- algorithm ---
def max_profit(prices):
    """Sell each day against the running minimum. O(n) time, O(1) space."""
    min_price = math.inf             # cheapest price so far
    best = 0                         # no trade at all earns 0
    for price in prices:
        if price < min_price:        # a cheaper day to have bought on
            min_price = price
        profit = price - min_price   # sell today, bought at the running minimum
        if profit > best:
            best = profit
    return best


# --- try it ---
print(max_profit([7, 1, 5, 3, 6, 4]))   # -> 5
print(max_profit([7, 6, 4, 3, 1]))      # -> 0
print(max_profit([2, 4, 1, 7]))         # -> 6
print(max_profit([]))                   # -> 0
```

22. Subsets

Choose, Recurse, Un-Choose · Backtracking · Tier 1 · python/022_subsets.py

Given an array of distinct integers, return every subset (the power set), each exactly once, in any order: [1, 2, 3] → [[], [1], [1, 2], [1, 2, 3], [1, 3], [2], [2, 3], [3]].

```python
# --- algorithm ---
def backtrack(nums, start, path, result):
    """Record path, then extend it by each index from start on. 2^n calls."""
    result.append(path.copy())          # every node of the tree is a subset
    for i in range(start, len(nums)):
        path.append(nums[i])            # choose nums[i]
        backtrack(nums, i + 1, path, result)    # only later indices below
        path.pop()                      # un-choose: path is as it was


def subsets(nums):
    """Start-index backtracking, one tree node per subset. O(n * 2^n)."""
    result = []
    backtrack(nums, 0, [], result)
    return result


# --- try it ---
result = subsets([1, 2, 3])
print(result)   # -> [[], [1], [1, 2], [1, 2, 3], [1, 3], [2], [2, 3], [3]]
print(subsets([7]))                     # -> [[], [7]]
print(subsets([]))                      # -> [[]]
print(len(subsets([1, 2, 3, 4, 5])))    # -> 32
```

23. Permutations

Backtracking with a Used Array · Backtracking · Tier 1 · python/023_permutations.py

Given an array of distinct integers, return all of its orderings, in any order: [1, 2, 3] → [[1, 2, 3], [1, 3, 2], [2, 1, 3], [2, 3, 1], [3, 1, 2], [3, 2, 1]].

```python
# --- algorithm ---
def backtrack(nums, used, path, result):
    """Fill the next slot with each unused value in turn. n! leaves."""
    if len(path) == len(nums):
        result.append(path.copy())      # every slot filled: one permutation
        return
    for i in range(len(nums)):
        if used[i]:
            continue                    # nums[i] already sits in the path
        used[i] = True                  # choose
        path.append(nums[i])
        backtrack(nums, used, path, result)
        path.pop()                      # un-choose, in reverse order
        used[i] = False


def permutations(nums):
    """Used-array backtracking over n slots. O(n * n!) time."""
    used = [False] * len(nums)
    result = []
    backtrack(nums, used, [], result)
    return result


# --- try it ---
result = permutations([1, 2, 3])
print(result[0:3])                  # -> [[1, 2, 3], [1, 3, 2], [2, 1, 3]]
print(result[3:6])                  # -> [[2, 3, 1], [3, 1, 2], [3, 2, 1]]
print(permutations([0, 1]))         # -> [[0, 1], [1, 0]]
print(permutations([5]))            # -> [[5]]
print(len(permutations([1, 2, 3, 4, 5])))   # -> 120
```

24. Binary Tree Level-Order Traversal

BFS One Level at a Time · BFS · Tier 1 · python/024_level_order_traversal.py

Given the root of a binary tree, return its values level by level, each level read left to right: the tree [3, 9, 20, None, None, 15, 7] → [[3], [9, 20], [15, 7]].

```python
from collections import deque      # popleft is O(1)


# --- helpers ---
# standard helpers TreeNode, build_tree: printed once under Standard Helpers


# --- algorithm ---
def level_order(root):
    """BFS; the queue length at the top of each round is one level. O(n)."""
    levels = []
    if root is None:
        return levels
    queue = deque([root])
    while queue:
        size = len(queue)              # snapshot: exactly this level's nodes
        level = []
        for i in range(size):
            node = queue.popleft()
            level.append(node.val)
            if node.left is not None:   # children join the back of the queue
                queue.append(node.left)
            if node.right is not None:
                queue.append(node.right)
        levels.append(level)           # the level is complete
    return levels


# --- try it ---
root = build_tree([3, 9, 20, None, None, 15, 7])
print(level_order(root))               # -> [[3], [9, 20], [15, 7]]
root = build_tree([1, 2, 3, 4, None, None, 5])
print(level_order(root))               # -> [[1], [2, 3], [4, 5]]
print(level_order(build_tree([1])))    # -> [[1]]
print(level_order(build_tree([])))     # -> []
```

25. Diameter and Max Path Sum

Postorder DFS That Returns a Height · DFS · Tier 1 · python/025_tree_diameter_max_path_sum.py

The diameter of a binary tree is the number of edges on its longest path; the max path sum is the largest total of values along any path of at least one node: [1, 2, 3, 4, 5] → diameter 3, [-10, 9, 20, None, None, 15, 7] → 42.

```python
from collections import deque      # popleft is O(1)


# --- helpers ---
# standard helpers TreeNode, build_tree: printed once under Standard Helpers


# --- algorithm ---
def diameter(root):
    """Longest path in edges: every node offers left + right. O(n)."""
    best = [0]                              # a box every call can update
    height(root, best)
    return best[0]


def height(node, best):
    """Nodes on the longest downward path from node; record the bend here."""
    if node is None:
        return 0
    left = height(node.left, best)
    right = height(node.right, best)
    best[0] = max(best[0], left + right)    # the path that bends at node
    return 1 + max(left, right)             # only one arm continues upward


def max_path_sum(root):
    """Largest path sum in a non-empty tree; negative arms are cut. O(n)."""
    best = [root.val]                       # one node alone is a path
    gain(root, best)
    return best[0]


def gain(node, best):
    """Best sum of a downward path from node; record the bend here."""
    if node is None:
        return 0
    left = max(gain(node.left, best), 0)    # a negative arm is cut off
    right = max(gain(node.right, best), 0)
    best[0] = max(best[0], node.val + left + right)
    return node.val + max(left, right)


# --- try it ---
root = build_tree([1, 2, 3, 4, 5])
print(diameter(root))                       # -> 3
root = build_tree([-10, 9, 20, None, None, 15, 7])
print(max_path_sum(root))                   # -> 42
root = build_tree([1, 2, None, 3, 4, 5, None, None, 6])
```

```
print(diameter(root))                        # -> 4
print(max_path_sum(build_tree([-3, -1, -2])))   # -> -1
print(diameter(build_tree([7])))             # -> 0
```

26. Lowest Common Ancestor

Postorder Search in a Binary Tree · DFS · Tier 1 · python/026_lowest_common_ancestor.py

Given a binary tree and two of its nodes p and q, return the deepest node that has both in its subtree (a node counts as its own ancestor): [3, 5, 1, 6, 2, 0, 8, None, None, 7, 4], p = 7, q = 8 → 3.

```python
from collections import deque      # popleft is O(1)


# --- helpers ---
# standard helpers TreeNode, build_tree: printed once under Standard Helpers


def find(root, val):
    """Return the node holding val, or None (values are unique)."""
    if root is None or root.val == val:
        return root
    left = find(root.left, val)
    if left is not None:
        return left
    return find(root.right, val)


# --- algorithm ---
def lowest_common_ancestor(root, p, q):
    """Postorder; report p, q or their meeting point upward. O(n) time."""
    if root is None or root is p or root is q:
        return root                    # nothing here, or a target itself
    left = lowest_common_ancestor(root.left, p, q)
    right = lowest_common_ancestor(root.right, p, q)
    if left is not None and right is not None:
        return root                    # one target on each side: the LCA
    if left is not None:
        return left                    # pass the single report upward
    return right


# --- try it ---
root = build_tree([3, 5, 1, 6, 2, 0, 8, None, None, 7, 4])
answer = lowest_common_ancestor(root, find(root, 7), find(root, 8))
print(answer.val)                      # -> 3
answer = lowest_common_ancestor(root, find(root, 6), find(root, 4))
print(answer.val)                      # -> 5
answer = lowest_common_ancestor(root, find(root, 5), find(root, 4))
print(answer.val)                      # -> 5
```

27. Validate a Binary Search Tree

Carry the Allowed Range Down · BST · Tier 1 · python/027_validate_bst.py

Given the root of a binary tree, decide whether it is a valid BST: every value in a node's left subtree is strictly smaller, every value in its right subtree strictly larger. [2, 1, 3] → True, [5, 4, 6, None, None, 3, 7] → False.

```
from collections import deque      # popleft is O(1)
import math                         # math.inf: no bound at the root


# --- helpers ---
# standard helpers TreeNode, build_tree: printed once under Standard Helpers


# --- algorithm ---
def is_valid_bst(root):
    """Start with no bounds; every node must fit its ancestors' range. O(n)."""
    return fits(root, -math.inf, math.inf)


def fits(node, low, high):
    """True if every value in this subtree is strictly between low and high."""
    if node is None:
        return True
    if node.val <= low or node.val >= high:
        return False                       # breaks a promise made higher up
    if not fits(node.left, low, node.val):  # left side: below node.val
        return False
    return fits(node.right, node.val, high)     # right side: above node.val


# --- try it ---
print(is_valid_bst(build_tree([2, 1, 3])))                      # -> True
print(is_valid_bst(build_tree([5, 4, 6, None, None, 3, 7])))    # -> False
print(is_valid_bst(build_tree([5, 1, 4, None, None, 3, 6])))    # -> False
print(is_valid_bst(build_tree([2, 2, 2])))                      # -> False
print(is_valid_bst(build_tree([])))                             # -> True
```

28. Depth-First Search on a Graph

Recursive and Iterative · DFS · Tier 1 · python/028_graph_dfs.py

Given an undirected graph with nodes 0..n-1 as an edge list, list the nodes in the order a depth-first search from start first reaches them: n = 5, edges [[0, 1], [0, 4], [0, 2], [1, 3], [3, 2]], start 0 → [0, 1, 3, 2, 4].

```python
# --- algorithm ---
def build_graph(n, edges):
    """Adjacency list: graph[u] lists u's neighbors in edge order. O(V + E)."""
    graph = []
    for i in range(n):
        graph.append([])
    for u, v in edges:
        graph[u].append(v)
        graph[v].append(u)                  # undirected: both directions
    return graph


def dfs_recursive(n, edges, start):
    """Visit a node, then dive into each unvisited neighbor. O(V + E)."""
    graph = build_graph(n, edges)
    visited = set()
    order = []
    visit(graph, start, visited, order)
    return order


def visit(graph, node, visited, order):
    """Mark node on entry, record it, recurse into unvisited neighbors."""
    visited.add(node)                       # mark first: cycles cannot re-enter
    order.append(node)
    for neighbor in graph[node]:
        if neighbor not in visited:
            visit(graph, neighbor, visited, order)


def dfs_iterative(n, edges, start):
    """Explicit stack; mark when popped to match the recursion. O(V + E)."""
    graph = build_graph(n, edges)
    visited = set()
    order = []
    stack = []
    stack.append(start)
    while stack:
        node = stack.pop()
        if node in visited:
            continue                        # a stale copy, reached another way
        visited.add(node)                   # mark when popped
        order.append(node)
        for neighbor in reversed(graph[node]):  # first neighbor ends on top
            if neighbor not in visited:
                stack.append(neighbor)
    return order
```

```
# --- try it ---
edges = [[0, 1], [0, 4], [0, 2], [1, 3], [3, 2]]
print(dfs_recursive(5, edges, 0))           # -> [0, 1, 3, 2, 4]
print(dfs_iterative(5, edges, 0))           # -> [0, 1, 3, 2, 4]
print(dfs_iterative(4, [[0, 1], [2, 3]], 0))    # -> [0, 1]
print(dfs_recursive(1, [], 0))              # -> [0]
```

29. Number of Islands

Grid Flood Fill · DFS · Tier 1 · python/029_number_of_islands.py

Given a grid of “1” (land) and “0” (water), count the islands: groups of land cells joined up, down, left or right, never diagonally: [“11000”, “11000”, “00100”, “00011”] → 3.

```python
# --- helpers ---
def make_grid(rows):
    """Turn a list of strings into a grid of one-character cells."""
    grid = []
    for row in rows:
        grid.append(list(row))
    return grid


# --- algorithm ---
def num_islands(grid):
    """Each '1' the scan meets starts a fill that sinks its island. O(R * C)."""
    count = 0
    for row in range(len(grid)):
        for col in range(len(grid[0])):
            if grid[row][col] == "1":
                count += 1                  # a new island starts here
                sink(grid, row, col)        # so it is never counted again
    return count


def sink(grid, start_row, start_col):
    """Explicit-stack DFS: turn every land cell of one island into water."""
    stack = []
    stack.append((start_row, start_col))
    while stack:
        row, col = stack.pop()
        if row < 0 or row >= len(grid) or col < 0 or col >= len(grid[0]):
            continue                        # stepped off the map
        if grid[row][col] != "1":
            continue                        # water, or land already sunk
        grid[row][col] = "0"                # sinking is the visited mark
        stack.append((row + 1, col))        # down, up, right, left
        stack.append((row - 1, col))
        stack.append((row, col + 1))
        stack.append((row, col - 1))


# --- try it ---
grid = make_grid(["11000", "11000", "00100", "00011"])
print(num_islands(grid))                    # -> 3
grid = make_grid(["11110", "11010", "11000", "00000"])
print(num_islands(grid))                    # -> 1
print(num_islands(make_grid(["101", "010", "101"])))    # -> 5
print(num_islands(make_grid(["0"])))        # -> 0
```

30. BFS Shortest Path

Unweighted Graphs, Layer by Layer · BFS · Tier 1 · python/030_bfs_shortest_path.py

Given an undirected graph on nodes 0..n-1 as an edge list, return a shortest path from source to target as a list of nodes, or [] if there is none: edges 0-1 1-2 2-3 3-4 0-5 5-4, source 0, target 4 → [0, 5, 4].

```python
from collections import deque      # popleft is O(1)


# --- algorithm ---
def shortest_path(n, edges, source, target):
    """BFS; the first time a node is reached is by a shortest path. O(V + E)."""
    graph = []
    for _ in range(n):
        graph.append([])
    for u, v in edges:
        graph[u].append(v)              # undirected: store both directions
        graph[v].append(u)
    dist = [-1] * n                     # -1 means not reached yet
    parent = [-1] * n                   # the node that first reached it
    dist[source] = 0
    queue = deque([source])
    while queue:
        node = queue.popleft()
        for neighbor in graph[node]:
            if dist[neighbor] == -1:    # first visit is a shortest one
                dist[neighbor] = dist[node] + 1     # mark when enqueued
                parent[neighbor] = node
                queue.append(neighbor)
    if dist[target] == -1:
        return []                       # target is in another component
    return build_path(parent, target)


def build_path(parent, target):
    """Follow parent pointers back to the source, then reverse. O(V)."""
    path = []
    node = target
    while node != -1:                   # only the source has no parent
        path.append(node)
        node = parent[node]
    return path[::-1]


# --- try it ---
edges = [[0, 1], [1, 2], [2, 3], [3, 4], [0, 5], [5, 4]]
print(shortest_path(6, edges, 0, 4))        # -> [0, 5, 4]
print(shortest_path(6, edges, 0, 3))        # -> [0, 1, 2, 3]
print(shortest_path(3, [[0, 1]], 0, 2))     # -> []
print(shortest_path(1, [], 0, 0))           # -> [0]
```

31. Kahn's Algorithm

Topological Sort by Peeling Indegree-Zero Nodes · Topological Sort · Tier 1 · python/031_kahn_topological_sort.py

Courses are 0..n-1, and a pair [a, b] means course b must come before course a. Return an order that takes every course, or [] if the pairs form a cycle: n = 6, [[3, 0], [3, 1], [4, 3], [2, 3], [5, 4], [5, 2]] → [0, 1, 3, 4, 2, 5].

```python
from collections import deque      # popleft is O(1)


# --- algorithm ---
def find_order(num_courses, prerequisites):
    """Take courses whose prerequisites are all done; a cycle never frees up.
    O(V + E) time and space."""
    graph = []
    for _ in range(num_courses):
        graph.append([])
    indegree = [0] * num_courses        # prerequisites not yet taken
    for course, before in prerequisites:
        graph[before].append(course)    # [a, b] is the edge b -> a
        indegree[course] += 1
    queue = deque()
    for course in range(num_courses):
        if indegree[course] == 0:       # free to take right away
            queue.append(course)
    order = []
    while queue:
        course = queue.popleft()
        order.append(course)
        for next_course in graph[course]:
            indegree[next_course] -= 1  # one prerequisite of it is done
            if indegree[next_course] == 0:
                queue.append(next_course)
    if len(order) < num_courses:
        return []                       # the rest wait on each other: a cycle
    return order


# --- try it ---
pairs = [[3, 0], [3, 1], [4, 3], [2, 3], [5, 4], [5, 2]]
print(find_order(6, pairs))                             # -> [0, 1, 3, 4, 2, 5]
print(find_order(4, [[1, 0], [2, 0], [3, 1], [3, 2]]))  # -> [0, 1, 2, 3]
print(find_order(3, [[1, 0], [2, 1], [1, 2]]))          # -> []
print(find_order(2, []))                                # -> [0, 1]
```

32. Union-Find

Path Compression + Union by Size · Connected Components · Tier 1 · python/032_union_find.py

Keep nodes 0..n-1 in disjoint groups: union(a, b) merges two groups, find(x) names the root of x's group, and a count tracks how many groups remain. Five nodes joined by [0, 1], [2, 3], [0, 2], [3, 1] (the last edge is redundant) form two groups: count_components(5, edges) → 2.

```python
# --- algorithm ---
class UnionFind:
    """Each group is a tree of parent pointers, named by its root."""

    def __init__(self, n):
        self.parent = list(range(n))    # every node starts as its own root
        self.size = [1] * n             # tree sizes, read only at roots
        self.count = n                  # number of separate groups

    def find(self, x):
        """Root of x; repoint the path straight at it. Near O(1) amortized."""
        if self.parent[x] != x:
            self.parent[x] = self.find(self.parent[x])    # path compression
        return self.parent[x]

    def union(self, a, b):
        """Hang the smaller tree under the bigger root; False if already one.
        Near O(1) amortized."""
        root_a = self.find(a)
        root_b = self.find(b)
        if root_a == root_b:
            return False                # already connected: a cycle edge
        if self.size[root_a] < self.size[root_b]:
            root_a, root_b = root_b, root_a     # root_a is the bigger tree
        self.parent[root_b] = root_a
        self.size[root_a] += self.size[root_b]
        self.count -= 1
        return True


def count_components(n, edges):
    """Union the ends of every edge; count what is left. Near O(n + E)."""
    uf = UnionFind(n)
    for a, b in edges:
        uf.union(a, b)
    return uf.count


def find_redundant_connection(edges):
    """Nodes are 1..n; the first edge inside one group closes the cycle.
    Near O(n) for n edges."""
    uf = UnionFind(len(edges) + 1)      # slot 0 is never used
    for a, b in edges:
        if not uf.union(a, b):
            return [a, b]
    return []
```

```python
# --- try it ---
print(count_components(5, [[0, 1], [2, 3], [0, 2], [3, 1]]))    # -> 2
print(count_components(4, []))                                  # -> 4
print(find_redundant_connection([[1, 2], [1, 3], [2, 3]]))      # -> [2, 3]
edges = [[1, 2], [2, 3], [3, 4], [1, 4], [1, 5]]
print(find_redundant_connection(edges))                         # -> [1, 4]
```

33. Dijkstra's Algorithm

Shortest Paths with Non-Negative Weights · Shortest Path · Tier 1 · python/033_dijkstra.py

A signal leaves node k of a network with nodes 1..n, and times[i] = [u, v, w] says it takes w to cross the directed edge u → v. Return the time until every node has the signal, or -1 if some node never gets it: n = 4, k = 1, [[1, 2, 4], [1, 3, 1], [3, 2, 2], [2, 4, 1], [3, 4, 5]] → 4.

```python
import heapq            # heappush / heappop keep the smallest at index 0
import math             # math.inf: no route known yet


# --- algorithm ---
def dijkstra(graph, source):
    """Pop the closest unsettled node; its distance is final. Relax its edges.
    O((V + E) log V) time."""
    dist = [math.inf] * len(graph)
    dist[source] = 0
    heap = []
    heapq.heappush(heap, (0, source))   # (distance so far, node)
    while heap:
        d, node = heapq.heappop(heap)
        if d > dist[node]:
            continue                    # stale entry: node was settled closer
        for neighbor, weight in graph[node]:
            if d + weight < dist[neighbor]:     # relax: a shorter route
                dist[neighbor] = d + weight
                heapq.heappush(heap, (dist[neighbor], neighbor))
    return dist


def network_delay_time(times, n, k):
    """Each node hears the signal at its shortest distance; take the largest.
    O((V + E) log V)."""
    graph = []
    for _ in range(n + 1):              # nodes are 1..n; slot 0 is unused
        graph.append([])
    for u, v, w in times:
        graph[u].append((v, w))
    dist = dijkstra(graph, k)
    slowest = 0
    for node in range(1, n + 1):
        if dist[node] == math.inf:
            return -1                   # this node never gets the signal
        slowest = max(slowest, dist[node])
    return slowest


# --- try it ---
times = [[1, 2, 4], [1, 3, 1], [3, 2, 2], [2, 4, 1], [3, 4, 5]]
print(network_delay_time(times, 4, 1))                                  # -> 4
print(network_delay_time([[2, 1, 1], [2, 3, 1], [3, 4, 1]], 4, 2))  # -> 2
print(network_delay_time([[1, 2, 1]], 2, 2))                            # -> -1
graph = [[], [(2, 4), (3, 1)], [(4, 1)], [(2, 2), (4, 5)], []]
print(dijkstra(graph, 1))                           # -> [inf, 0, 3, 1, 4]
```

34. Trie

Insert, Search, Starts With · Tries · Tier 1 · python/034_trie.py

Store words so that search(word) says whether that exact word was inserted and starts_with(prefix) whether any inserted word begins with prefix: after inserting apple, app, apt and bat, search("ap") → False and starts_with("ap") → True.

```python
# --- helpers ---
class TrieNode:
    def __init__(self):
        self.children = {}              # letter -> TrieNode
        self.is_end = False             # True if a word ends at this node


# --- algorithm ---
class Trie:
    def __init__(self):
        self.root = TrieNode()

    def insert(self, word):
        """Walk down the letters, creating missing children. O(L)."""
        node = self.root
        for ch in word:
            if ch not in node.children:
                node.children[ch] = TrieNode()
            node = node.children[ch]
        node.is_end = True              # flag the node where the word stops

    def walk(self, prefix):
        """Follow the letters of prefix; the last node, or None. O(L)."""
        node = self.root
        for ch in prefix:
            if ch not in node.children:
                return None             # the path breaks off
            node = node.children[ch]
        return node

    def search(self, word):
        """The path must exist and end on a flagged node. O(L)."""
        node = self.walk(word)
        return node is not None and node.is_end

    def starts_with(self, prefix):
        """The path existing is enough. O(L)."""
        return self.walk(prefix) is not None


# --- try it ---
trie = Trie()
for word in ["apple", "app", "apt", "bat"]:
    trie.insert(word)
print(trie.search("app"))           # -> True
print(trie.search("ap"))            # -> False
print(trie.starts_with("ap"))       # -> True
print(trie.search("apl"))           # -> False
```

```
print(trie.starts_with("c"))        # -> False
```

35. LRU Cache

Hash Map + Doubly Linked List · Design · Tier 1 · python/035_lru_cache.py

Design a cache with a fixed capacity: get(key) returns the value or -1, and put(key, value) inserts or updates; a put that overflows the capacity evicts the least recently used key. Capacity 2: put(1, 10), put(2, 20), get(1), put(3, 30), get(2) → 10, -1.

```python
# --- helpers ---
class Node:
    def __init__(self, key, value):
        self.key = key              # kept so eviction can clean the map
        self.value = value
        self.prev = None
        self.next = None


# --- algorithm ---
class LRUCache:
    """Map key -> node; the list runs from most to least recently used."""

    def __init__(self, capacity):
        self.capacity = capacity
        self.nodes = {}             # key -> node
        self.head = Node(0, 0)      # sentinel: head.next is most recent
        self.tail = Node(0, 0)      # sentinel: tail.prev is least recent
        self.head.next = self.tail
        self.tail.prev = self.head

    def remove(self, node):
        """Unlink node: its two neighbors point past it. O(1)."""
        node.prev.next = node.next
        node.next.prev = node.prev

    def add_front(self, node):
        """Splice node in right after head, the most recent spot. O(1)."""
        node.prev = self.head
        node.next = self.head.next
        self.head.next.prev = node
        self.head.next = node

    def get(self, key):
        """Return the value and make the key most recent; -1 if absent. O(1)."""
        if key not in self.nodes:
            return -1
        node = self.nodes[key]
        self.remove(node)           # a read counts as a use
        self.add_front(node)
        return node.value

    def put(self, key, value):
        """Insert or update the key as most recent; if that overflows the
        capacity, evict the node before tail. O(1)."""
        if key in self.nodes:
            node = self.nodes[key]
            node.value = value
```

```
            self.remove(node)       # it moves to the front below
        else:
            node = Node(key, value)
            self.nodes[key] = node
        self.add_front(node)
        if len(self.nodes) > self.capacity:
            lru = self.tail.prev    # least recently used
            self.remove(lru)
            del self.nodes[lru.key]


# --- try it ---
cache = LRUCache(2)
cache.put(1, 10)
cache.put(2, 20)
print(cache.get(1))         # -> 10
cache.put(3, 30)            # evicts key 2
print(cache.get(2))         # -> -1
cache.put(4, 40)            # evicts key 1
print(cache.get(1))         # -> -1
print(cache.get(3))         # -> 30
print(cache.get(4))         # -> 40
```

Tier 2 · Should Know Well

The next thirty-five deepen every topic of the first tier and add the classic harder variants: partition-based selection, the histogram stack, expression evaluation, the greedy exchange arguments, grid backtracking, BFS on implicit graphs, topological order by DFS, KMP. They come up often enough that each one deserves a careful read and a run of its code.

36. Hash Map with Separate Chaining

Collisions Live in Buckets · Hash Map Internals · Tier 2 · python/036_hash_map_chaining.py

Design a hash map without a built-in dict: put(key, value) inserts or overwrites, get(key) returns the value or -1, remove(key) deletes the key. With 8 buckets, keys 1 and 9 share bucket 1: put(1, 10), put(9, 90), get(9) → 90.

```python
# --- algorithm ---
class ChainedHashMap:
    def __init__(self, capacity=8):
        self.capacity = capacity
        self.size = 0                       # number of keys stored
        self.buckets = []
        for i in range(capacity):
            self.buckets.append([])         # one empty chain per bucket

    def bucket_of(self, key):
        """The chain that holds key: bucket hash(key) mod capacity. O(1)."""
        return self.buckets[hash(key) % self.capacity]

    def put(self, key, value):
        """Overwrite the key in its chain or append a new pair. Average O(1)."""
        bucket = self.bucket_of(key)
        for pair in bucket:
            if pair[0] == key:              # same key: overwrite the value
                pair[1] = value
                return
        bucket.append([key, value])         # new key: the chain grows by one
        self.size += 1
        if self.size / self.capacity > 0.75:
            self.resize()                   # load factor too high: grow

    def get(self, key):
        """Scan only the key's chain. Average O(1), worst O(n)."""
        bucket = self.bucket_of(key)
        for pair in bucket:
            if pair[0] == key:              # compare keys, not hashes
                return pair[1]
        return -1

    def remove(self, key):
        """Unlink the key's pair from its chain if present. Average O(1)."""
        bucket = self.bucket_of(key)
        for pair in bucket:
            if pair[0] == key:
                bucket.remove(pair)
                self.size -= 1
                return

    def resize(self):
        """Double the bucket array and rehash every pair. O(n), but rare."""
        old_buckets = self.buckets
        self.capacity = 2 * self.capacity
        self.buckets = []
        for i in range(self.capacity):
```

```
            self.buckets.append([])
        for bucket in old_buckets:
            for pair in bucket:
                new_bucket = self.bucket_of(pair[0])    # uses the new capacity
                new_bucket.append(pair)


# --- try it ---
table = ChainedHashMap()
table.put(1, 10)
table.put(9, 90)                            # 9 mod 8 = 1: same bucket as 1
table.put(1, 11)                            # overwrite
print(table.get(1), table.get(9), table.get(2))     # -> 11 90 -1
table.remove(9)
print(table.get(9), table.size)             # -> -1 1
for key in range(2, 8):
    table.put(key, key * 10)
print(table.capacity, table.size, table.get(7))     # -> 16 7 70
```

37. Trapping Rain Water

Two Pointers with Running Walls · Two Pointers · Tier 2 · python/037_trapping_rain_water.py

Given the heights of bars of width 1, return how many units of rain water they trap between them: [0, 1, 0, 2, 1, 0, 1, 3, 2, 1, 2, 1] → 6.

```python
# --- algorithm ---
def trap(height):
    """Water at i is min(tallest left, tallest right) - height[i]; the side
    with the lower running wall is settled, so move it. O(n), O(1) space."""
    left = 0
    right = len(height) - 1
    left_max = 0                    # tallest bar in height[0..left]
    right_max = 0                   # tallest bar in height[right..n-1]
    water = 0
    while left < right:
        left_max = max(left_max, height[left])
        right_max = max(right_max, height[right])
        if left_max < right_max:    # right wall taller: settle left
            water += left_max - height[left]
            left += 1
        else:                       # left wall as tall: settle right
            water += right_max - height[right]
            right -= 1
    return water


# --- try it ---
print(trap([0, 1, 0, 2, 1, 0, 1, 3, 2, 1, 2, 1]))   # -> 6
print(trap([4, 2, 0, 3, 2, 5]))                     # -> 9
print(trap([2, 0, 2]))                              # -> 2
print(trap([1, 2, 3]))                              # -> 0
print(trap([]))                                     # -> 0
```

38. Dutch National Flag

Three-Way Partition in One Pass · Two Pointers · Tier 2 · python/038_dutch_national_flag.py

Given an array holding only 0s, 1s and 2s (three colors), sort it in place in a single pass with O(1) extra space: [2, 0, 2, 1, 1, 0] → [0, 0, 1, 1, 2, 2].

```python
# --- algorithm ---
def sort_colors(nums):
    """Grow the 0s from the left and the 2s from the right; the 1s are left
    in the middle. One pass, O(n) time, O(1) space."""
    low = 0                         # nums[0..low-1] are 0s
    mid = 0                         # nums[low..mid-1] are 1s
    high = len(nums) - 1            # nums[high+1..n-1] are 2s
    while mid <= high:              # nums[mid..high] is still unseen
        if nums[mid] == 0:
            nums[low], nums[mid] = nums[mid], nums[low]
            low += 1
            mid += 1                # the block of 1s shifts right
        elif nums[mid] == 1:
            mid += 1
        else:
            nums[mid], nums[high] = nums[high], nums[mid]
            high -= 1               # mid stays: this value is unseen


# --- try it ---
colors = [2, 0, 2, 1, 1, 0]
sort_colors(colors)
print(colors)                       # -> [0, 0, 1, 1, 2, 2]
colors = [1, 2, 0]
sort_colors(colors)
print(colors)                       # -> [0, 1, 2]
colors = [2, 2, 0, 0]
sort_colors(colors)
print(colors)                       # -> [0, 0, 2, 2]
```

39. Search in a Rotated Sorted Array

One Half Is Always Sorted · Binary Search · Tier 2 · python/039_search_rotated_sorted_array.py

A sorted array of distinct values was rotated at an unknown pivot. Return the index of the target, or -1 if it is absent, in O(log n) time: [6, 7, 0, 1, 2, 4, 5], target 0 → 2.

```python
# --- algorithm ---
def search_rotated(nums, target):
    """Binary search; the sorted half tells which side can hold the target.
    O(log n) time, O(1) space."""
    lo = 0
    hi = len(nums) - 1                          # closed window [lo, hi]
    while lo <= hi:
        mid = (lo + hi) // 2
        if nums[mid] == target:
            return mid
        if nums[lo] <= nums[mid]:               # left half is sorted
            if nums[lo] <= target and target < nums[mid]:
                hi = mid - 1                    # target inside the left half
            else:
                lo = mid + 1                    # so it must be on the right
        else:                                   # right half is sorted
            if nums[mid] < target and target <= nums[hi]:
                lo = mid + 1                    # target inside the right half
            else:
                hi = mid - 1                    # so it must be on the left
    return -1


# --- try it ---
print(search_rotated([6, 7, 0, 1, 2, 4, 5], 0))     # -> 2
print(search_rotated([4, 5, 6, 7, 0, 1, 2], 0))     # -> 4
print(search_rotated([4, 5, 6, 7, 0, 1, 2], 3))     # -> -1
print(search_rotated([3, 1], 1))                    # -> 1
```

40. Median of Two Sorted Arrays

Binary Search the Partition · Binary Search · Tier 2 · python/040_median_two_sorted_arrays.py

Given two sorted arrays with at least one value between them, return the median of all their values together in O(log(m + n)) time: [1, 3, 8, 9, 15] and [7, 11, 18, 19, 21, 25] → 11.

```python
import math                          # math.inf: a cut at the edge sees infinity


# --- algorithm ---
def value_at(arr, k):
    """arr[k], or -inf before the start and +inf past the end. O(1)."""
    if k < 0:
        return -math.inf
    if k >= len(arr):
        return math.inf
    return arr[k]


def find_median(a, b):
    """Binary search the cut i in the shorter array; b's cut j follows.
    O(log min(m, n)) time, O(1) space."""
    if len(a) > len(b):
        a, b = b, a                  # search the shorter array
    m = len(a)
    n = len(b)
    half = (m + n + 1) // 2          # size of the left part
    lo = 0
    hi = m
    while lo <= hi:
        i = (lo + hi) // 2           # a[:i] goes to the left part
        j = half - i                 # b[:j] fills the rest of it
        a_left = value_at(a, i - 1)
        a_right = value_at(a, i)
        b_left = value_at(b, j - 1)
        b_right = value_at(b, j)
        if a_left <= b_right and b_left <= a_right:    # every left <= right
            if (m + n) % 2 == 1:
                return max(a_left, b_left)
            return (max(a_left, b_left) + min(a_right, b_right)) / 2
        if a_left > b_right:
            hi = i - 1               # too much of a on the left: cut a earlier
        else:
            lo = i + 1               # too little of a on the left: cut a later


# --- try it ---
print(find_median([1, 3, 8, 9, 15], [7, 11, 18, 19, 21, 25]))    # -> 11
print(find_median([1, 3], [2]))                                  # -> 2
print(find_median([1, 2], [3, 4]))                               # -> 2.5
print(find_median([], [4]))                                      # -> 4
```

41. Heapify

Build a Heap in O(n) with Sift-Down · Heaps · Tier 2 · python/041_heapify.py

Rearrange an array in place into a min-heap, a tree stored level by level in which every parent is at most its children, then push and pop on it by hand: [9, 5, 6, 2, 3, 7, 1] → [1, 2, 6, 5, 3, 7, 9].

```python
import heapq   # the library min-heap, used here only to compare


# --- algorithm ---
def sift_down(heap, i, n):
    """Swap heap[i] with its smaller child until no child is smaller; only
    heap[0..n-1] is the heap. O(log n)."""
    while True:
        smallest = i
        left = 2 * i + 1
        right = 2 * i + 2
        if left < n and heap[left] < heap[smallest]:
            smallest = left
        if right < n and heap[right] < heap[smallest]:
            smallest = right
        if smallest == i:               # no child is smaller: done
            return
        heap[i], heap[smallest] = heap[smallest], heap[i]
        i = smallest


def heapify(heap):
    """Sift down every parent, the last parent first. O(n) in total."""
    n = len(heap)
    for i in range(n // 2 - 1, -1, -1):  # last parent, back to the root
        sift_down(heap, i, n)


def sift_up(heap, i):
    """Swap heap[i] with its parent while it is smaller. O(log n)."""
    while i > 0:
        parent = (i - 1) // 2
        if heap[parent] <= heap[i]:     # the parent is no larger: done
            return
        heap[i], heap[parent] = heap[parent], heap[i]
        i = parent


def heap_push(heap, value):
    """Add the value as the last leaf, then sift it up. O(log n)."""
    heap.append(value)
    sift_up(heap, len(heap) - 1)


def heap_pop(heap):
    """Swap the root with the last leaf, cut it off, sift down. O(log n)."""
    last = len(heap) - 1
    heap[0], heap[last] = heap[last], heap[0]
    top = heap.pop()                    # remove the last slot: the old root
```

```
    sift_down(heap, 0, len(heap))
    return top


# --- try it ---
nums = [9, 5, 6, 2, 3, 7, 1]
heapify(nums)
print(nums)                             # -> [1, 2, 6, 5, 3, 7, 9]
heap_push(nums, 4)
print(nums)                             # -> [1, 2, 6, 4, 3, 7, 9, 5]
print(heap_pop(nums))                   # -> 1
print(nums)                             # -> [2, 3, 6, 4, 5, 7, 9]
same = [9, 5, 6, 2, 3, 7, 1]
heapq.heapify(same)                     # the library: same array this time
print(same)                             # -> [1, 2, 6, 5, 3, 7, 9]
```

42. Quickselect

Kth Largest in Expected O(n) · Heaps · Tier 2 · python/042_quickselect.py

Find the kth largest value of an unsorted array (duplicates count) without sorting it; the array is reordered in place: [3, 2, 1, 5, 6, 4], k = 2 → 5.

```python
import random   # random.randint(a, b) can return a or b too


# --- algorithm ---
def partition(nums, lo, hi):
    """Lomuto partition of nums[lo..hi] around a random pivot; return the
    pivot's final index. O(hi - lo) time."""
    pick = random.randint(lo, hi)
    nums[pick], nums[hi] = nums[hi], nums[pick]  # park the pivot at the end
    pivot = nums[hi]
    store = lo                          # invariant: nums[lo..store-1] < pivot
    for i in range(lo, hi):
        if nums[i] < pivot:             # a small value joins the left region
            nums[store], nums[i] = nums[i], nums[store]
            store += 1
    nums[store], nums[hi] = nums[hi], nums[store]  # pivot between the regions
    return store


def kth_largest(nums, k):
    """Partition, then keep only the side that holds index n - k.
    Expected O(n) time on distinct values, O(1) extra space."""
    target = len(nums) - k              # the answer's index in sorted order
    lo = 0
    hi = len(nums) - 1
    while True:
        p = partition(nums, lo, hi)
        if p == target:
            return nums[p]
        elif p < target:
            lo = p + 1                  # the answer is right of the pivot
        else:
            hi = p - 1                  # the answer is left of the pivot


# --- try it ---
print(kth_largest([3, 2, 1, 5, 6, 4], 2))            # -> 5
print(kth_largest([3, 2, 3, 1, 2, 4, 5, 5, 6], 4))   # -> 4
print(kth_largest([7], 1))                           # -> 7
print(kth_largest([-2, 0, -5], 3))                   # -> -5
```

43. Bucket Sort

Top K Frequent by Frequency Buckets · Sorting · Tier 2 · python/043_bucket_sort.py

Return the k most frequent values of an array in any order (the answer is unique), in O(n) instead of sorting by frequency: [1, 1, 1, 2, 2, 3], k = 2 → [1, 2].

```python
# --- algorithm ---
def top_k_frequent(nums, k):
    """Put each value in the bucket of its frequency, then read the buckets
    from the highest frequency down. O(n) time and space."""
    count = {}                                # value -> frequency
    for value in nums:
        count[value] = count.get(value, 0) + 1
    buckets = []                              # index = frequency
    for freq in range(len(nums) + 1):         # a frequency is at most n
        buckets.append([])
    for value in count:
        buckets[count[value]].append(value)
    result = []
    for freq in range(len(nums), 0, -1):      # most frequent first
        for value in buckets[freq]:
            result.append(value)
            if len(result) == k:
                return result
    return result


# --- try it ---
print(top_k_frequent([1, 1, 1, 2, 2, 3], 2))        # -> [1, 2]
print(top_k_frequent([4, 1, -1, 2, -1, 2, 3], 2))   # -> [-1, 2]
print(top_k_frequent([7, 7, 7], 1))                 # -> [7]
print(top_k_frequent([5], 1))                       # -> [5]
```

44. Largest Rectangle in Histogram

Monotonic Increasing Stack · Stacks · Tier 2 · python/044_largest_rectangle_histogram.py

Given bar heights of a histogram whose bars all have width 1, return the area of the largest rectangle that fits inside it: [2, 1, 5, 6, 2, 3] → 10 (the bars 5 and 6, height 5, width 2).

```python
# --- algorithm ---
def largest_rectangle_area(heights):
    """Stack of indices with rising heights; a shorter bar ends the popped
    bar's rectangle on the right. O(n) time, O(n) space."""
    bars = heights + [0]            # the 0 sentinel flushes the stack
    stack = []                      # indices; heights never fall
    best = 0
    for i in range(len(bars)):
        while len(stack) > 0 and bars[stack[-1]] > bars[i]:
            height = bars[stack.pop()]      # its rectangle ends at i - 1
            if len(stack) > 0:
                left = stack[-1] + 1        # just right of the new stack top
            else:
                left = 0                    # empty stack: reaches index 0
            width = i - left
            best = max(best, height * width)
        stack.append(i)
    return best


# --- try it ---
print(largest_rectangle_area([2, 1, 5, 6, 2, 3]))  # -> 10
print(largest_rectangle_area([2, 4]))              # -> 4
print(largest_rectangle_area([1, 2, 3]))           # -> 4
print(largest_rectangle_area([3, 3, 3]))           # -> 9
```

45. Basic Calculator

Evaluate + - * / and Parentheses with a Stack · Stacks · Tier 2 · python/045_basic_calculator.py

Evaluate a valid expression of non-negative integers, + - * /, parentheses and spaces, with the usual precedence; / truncates toward zero and a minus may open the expression or a group: “2*(3+4)-10/3” → 11.

```python
# --- algorithm ---
def calculate(s):
    """Evaluate the whole string. O(n) time, O(n) space."""
    value, end = evaluate(s, 0)
    return value


def evaluate(s, i):
    """Evaluate from index i up to the matching ')' or the end of s.
    Returns (value, next index to read). Linear in what it reads."""
    stack = []                  # signed terms of this level; value = their sum
    num = 0
    op = "+"                    # the operator in front of num
    while i < len(s):
        ch = s[i]
        i += 1
        if ch.isdigit():
            num = num * 10 + int(ch)
        elif ch == "(":
            num, i = evaluate(s, i)     # a whole group acts as one number
        elif ch in "+-*/)":
            apply_op(stack, op, num)
            if ch == ")":
                return sum(stack), i
            op = ch
            num = 0
    apply_op(stack, op, num)
    return sum(stack), i


def apply_op(stack, op, num):
    """Fold num into the terms using the operator in front of it. O(1)."""
    if op == "+":
        stack.append(num)
    elif op == "-":
        stack.append(-num)
    elif op == "*":
        stack.append(stack.pop() * num)     # * reaches back one term only
    else:
        stack.append(int(stack.pop() / num))    # int() truncates toward 0


# --- try it ---
print(calculate("2*(3+4)-10/3"))           # -> 11
print(calculate(" 3+5 / 2 "))              # -> 5
print(calculate("(1+(4+5+2)-3)+(6+8)"))    # -> 23
print(calculate("-(2 + 3)"))               # -> -5
```

46. Sliding Window Maximum

Monotonic Deque · Queues · Tier 2 · python/046_sliding_window_maximum.py

Given an array and a window size k, return the maximum of every window of k consecutive values as the window slides from left to right: [9, 7, 8, 3, 2, 5, 1], k 3 → [9, 8, 8, 5, 5].

```python
from collections import deque      # popleft is O(1)


# --- algorithm ---
def max_sliding_window(nums, k):
    """Deque of indices whose values decrease from front to back; the front
    is the window max. Each index enters and leaves once: O(n) time."""
    window = deque()               # indices of the candidates
    result = []
    for i in range(len(nums)):
        while len(window) > 0 and nums[window[-1]] <= nums[i]:
            window.pop()           # older and no larger: never needed again
        window.append(i)
        if window[0] <= i - k:
            window.popleft()       # the front slid out of the window
        if i >= k - 1:
            result.append(nums[window[0]])  # the front holds the window max
    return result


# --- try it ---
print(max_sliding_window([9, 7, 8, 3, 2, 5, 1], 3))    # -> [9, 8, 8, 5, 5]
nums = [1, 3, -1, -3, 5, 3, 6, 7]
print(max_sliding_window(nums, 3))                     # -> [3, 3, 5, 5, 6, 7]
print(max_sliding_window([4, 4, 4], 2))                # -> [4, 4]
print(max_sliding_window([5], 1))                      # -> [5]
```

47. Jump Game

Track the Farthest Reach · Greedy · Tier 2 · python/047_jump_game.py

In a non-empty array, nums[i] is the longest jump allowed from index i (any shorter jump is allowed too). Starting at index 0, can you reach the last index? [2, 3, 1, 1, 4] → True, [3, 2, 1, 0, 4] → False.

```python
# --- algorithm ---
def can_jump(nums):
    """Grow the block [0, farthest] while i is in it. O(n) time, O(1) space."""
    farthest = 0                        # indices 0..farthest are reachable
    i = 0
    while i <= farthest:                # i is a spot we can really stand on
        farthest = max(farthest, i + nums[i])
        if farthest >= len(nums) - 1:   # the reach covers the last index
            return True
        i += 1
    return False                        # walked past the reach: a wall


# --- try it ---
print(can_jump([2, 3, 1, 1, 4]))    # -> True
print(can_jump([3, 2, 1, 0, 4]))    # -> False
print(can_jump([0]))                # -> True
print(can_jump([2, 0, 0]))          # -> True
print(can_jump([1, 0, 1]))          # -> False
```

48. Jump Game II

Greedy BFS over Reach Levels · Greedy · Tier 2 · python/048_jump_game_ii.py

nums[i] is the longest jump allowed from index i, and the last index is known to be reachable. Return the fewest jumps from index 0 to the last index: [2, 3, 1, 1, 4] → 2 (index 0 → 1 → 4).

```python
# --- algorithm ---
def min_jumps(nums):
    """BFS whose levels are intervals; count levels. O(n) time, O(1) space."""
    jumps = 0
    current_end = 0                # right edge of the current level
    farthest = 0                   # right edge of the next level
    for i in range(len(nums) - 1):  # never jump from the last index
        farthest = max(farthest, i + nums[i])
        if i == current_end:        # this level is used up: jump again
            jumps += 1
            current_end = farthest  # the next level ends here
    return jumps


# --- try it ---
print(min_jumps([2, 3, 1, 1, 4]))   # -> 2
print(min_jumps([2, 3, 0, 1, 4]))   # -> 2
print(min_jumps([1, 1, 1, 1]))      # -> 3
print(min_jumps([0]))               # -> 0
```

49. Gas Station

Restart After Every Deficit · Greedy · Tier 2 · python/049_gas_station.py

Station i on a circle gives gas[i] fuel; driving on to the next one burns cost[i]. Starting with an empty tank, return the station a full lap can start from (the input promises at most one), or -1: gas [4, 1, 2, 5, 1], cost [3, 3, 3, 1, 2] → 3.

```python
# --- algorithm ---
def can_complete_circuit(gas, cost):
    """Check the totals, then skip past every deficit. O(n) time, O(1) space."""
    if sum(gas) < sum(cost):        # not enough fuel for any lap
        return -1
    start = 0
    tank = 0                        # fuel left since leaving start
    for i in range(len(gas)):
        tank += gas[i] - cost[i]
        if tank < 0:                # stranded: start..i are all out
            start = i + 1
            tank = 0
    return start


# --- try it ---
print(can_complete_circuit([4, 1, 2, 5, 1], [3, 3, 3, 1, 2]))   # -> 3
print(can_complete_circuit([1, 2, 3, 4, 5], [3, 4, 5, 1, 2]))   # -> 3
print(can_complete_circuit([2, 3, 4], [3, 4, 3]))               # -> -1
print(can_complete_circuit([5], [4]))                           # -> 0
```

50. Interval Scheduling

Sort by End, Keep What Fits · Greedy · Tier 2 · python/050_interval_scheduling.py

Keep as many intervals [start, end] as possible with no two overlapping (ends that only touch, like [1, 3] and [3, 5], are fine); the fewest removals is the rest: [[1, 3], [2, 4], [3, 5], [1, 8], [6, 7]] → keep 3, remove 2.

```python
import math                          # math.inf


# --- algorithm ---
def max_non_overlapping(intervals):
    """Sort by end, keep whatever fits after the last kept end. O(n log n)."""
    by_end = []
    for start, end in intervals:
        by_end.append((end, start))     # end first: the sort orders by end
    by_end.sort()
    kept = 0
    last_end = -math.inf            # end of the last kept interval
    for end, start in by_end:
        if start >= last_end:       # fits after the last kept one
            kept += 1
            last_end = end
    return kept


def min_removals(intervals):
    """Remove everything the greedy did not keep (LC 435). O(n log n)."""
    return len(intervals) - max_non_overlapping(intervals)


# --- try it ---
intervals = [[1, 3], [2, 4], [3, 5], [1, 8], [6, 7]]
print(max_non_overlapping(intervals))           # -> 3
print(min_removals(intervals))                  # -> 2
print(min_removals([[1, 2], [1, 2], [1, 2]]))   # -> 2
print(max_non_overlapping([]))                  # -> 0
```

51. Task Scheduler

Frame the Most Frequent Task · Greedy · Tier 2 · python/051_task_scheduler.py

Each task takes one unit of time, and two runs of the same task need at least n units between them; the CPU may idle. Return the fewest units for all tasks: [“A”, “A”, “A”, “B”, “B”, “B”], n = 2 → 8 (A B idle A B idle A B).

```python
# --- algorithm ---
def least_interval(tasks, n):
    """The most frequent task frames the schedule. O(len(tasks)) time."""
    counts = {}
    for task in tasks:
        counts[task] = counts.get(task, 0) + 1
    max_count = 0
    for task in counts:
        max_count = max(max_count, counts[task])
    tied = 0                        # tasks that also reach max_count
    for task in counts:
        if counts[task] == max_count:
            tied += 1
    frame = (max_count - 1) * (n + 1) + tied    # full rows, then a last row
    return max(frame, len(tasks))   # overflow: rows widen, no idles


# --- try it ---
print(least_interval(["A", "A", "A", "B", "B", "B"], 2))      # -> 8
print(least_interval(["A", "A", "A", "B", "B", "B"], 0))      # -> 6
print(least_interval(["A", "A", "A", "B", "B", "C"], 2))      # -> 7
overflow = ["A", "A", "A", "B", "B", "B", "C", "C", "D", "D"]
print(least_interval(overflow, 2))                            # -> 10
```

52. Partition Labels

Extend to the Last Occurrence · Greedy · Tier 2 · python/052_partition_labels.py

Cut a string into as many parts as possible so that each letter appears in at most one part, and return the part sizes in order: “ababcbacadefegdehijhklij” → [9, 7, 8].

```python
# --- algorithm ---
def partition_labels(s):
    """Record each letter's last index; cut where i meets part_end. O(n)."""
    last = {}                       # letter -> index of its last copy
    for i in range(len(s)):
        last[s[i]] = i
    sizes = []
    start = 0                       # first index of the current part
    part_end = 0                    # the part must reach at least here
    for i in range(len(s)):
        part_end = max(part_end, last[s[i]])    # this letter drags it along
        if i == part_end:           # nothing inside reaches past i: cut
            sizes.append(i - start + 1)
            start = i + 1
    return sizes


# --- try it ---
print(partition_labels("ababcbacadefegdehijhklij"))   # -> [9, 7, 8]
print(partition_labels("eccbbbbdec"))                 # -> [10]
print(partition_labels("abac"))                       # -> [3, 1]
print(partition_labels(""))                           # -> []
```

53. Combination Sum

Unlimited Reuse with a Start Index · Backtracking · Tier 2 · python/053_combination_sum.py

Given distinct positive integers and a target, return every combination of them that sums to the target; a number may be used any number of times, and order does not matter: [2, 3, 6, 7], target 7 → [[2, 2, 3], [7]].

```python
# --- algorithm ---
def backtrack(candidates, start, remaining, path, result):
    """Extend path from index start on, reusing i; break at an overshoot."""
    if remaining == 0:
        result.append(path.copy())      # exact hit: one combination
        return
    for i in range(start, len(candidates)):
        if candidates[i] > remaining:
            break                       # sorted: every later one is bigger
        path.append(candidates[i])      # choose
        # pass i, not i + 1: the same candidate may be chosen again
        backtrack(candidates, i, remaining - candidates[i], path, result)
        path.pop()                      # un-choose


def combination_sum(candidates, target):
    """Sort, then start-index backtracking that may reuse i. Exponential."""
    candidates = sorted(candidates)     # makes the break above valid
    result = []
    backtrack(candidates, 0, target, [], result)
    return result


# --- try it ---
print(combination_sum([2, 3, 6, 7], 7))   # -> [[2, 2, 3], [7]]
print(combination_sum([2, 3, 5], 8))      # -> [[2, 2, 2, 2], [2, 3, 3], [3, 5]]
print(combination_sum([7, 3, 2], 7))      # -> [[2, 2, 3], [7]]
print(combination_sum([2], 1))            # -> []
```

54. Word Search

Grid Backtracking with In-Place Marks · Backtracking · Tier 2 · python/054_word_search.py

Given a grid of letters and a word, decide whether the word can be spelled by a path of horizontally or vertically adjacent cells, each used at most once: grid ABCE / SFCS / ADEE, word “ABCCED” → True.

```python
# --- algorithm ---
def dfs(board, word, row, col, k):
    """Can word[k:] be spelled from cell (row, col) on? Marks its own path."""
    if k == len(word):
        return True                     # every letter matched
    if row < 0 or row >= len(board) or col < 0 or col >= len(board[0]):
        return False                    # off the grid
    if board[row][col] != word[k]:
        return False                    # wrong letter, or a '#' on the path
    board[row][col] = "#"               # mark: this cell is on the path
    found = False
    for d_row, d_col in [(1, 0), (-1, 0), (0, 1), (0, -1)]:
        if dfs(board, word, row + d_row, col + d_col, k + 1):
            found = True
            break
    board[row][col] = word[k]           # unmark on the way back
    return found


def exist(board, word):
    """Try every cell as the first letter. O(m * n * 3^L) worst case."""
    for row in range(len(board)):
        for col in range(len(board[0])):
            if dfs(board, word, row, col, 0):
                return True
    return False


# --- try it ---
board = [list("ABCE"), list("SFCS"), list("ADEE")]   # rows of letters
print(exist(board, "ABCCED"))   # -> True
print(exist(board, "SEE"))      # -> True
print(exist(board, "ABCB"))     # -> False
print(board[0])                 # -> ['A', 'B', 'C', 'E']
```

55. Serialize and Deserialize a Binary Tree

Preorder with Null Markers · DFS · Tier 2 · python/055_serialize_binary_tree.py

Turn a binary tree into a string and the string back into the same tree: write the values in preorder, '#' for every missing child, separated by commas: [1, 2, 3, None, None, 4, 5] → "1,2,#,#,3,4,#,#,5,#,#".

```python
from collections import deque      # popleft is O(1)


# --- helpers ---
# standard helpers TreeNode, build_tree, tree_to_list: printed once under Standard Helpers


# --- algorithm ---
def serialize(root):
    """Preorder with '#' for each missing child, joined by commas. O(n)."""
    tokens = []
    write(root, tokens)
    return ",".join(tokens)


def write(node, tokens):
    """Append the node's value, then its left subtree, then its right."""
    if node is None:
        tokens.append("#")                 # the marker closes a branch
        return
    tokens.append(str(node.val))
    write(node.left, tokens)
    write(node.right, tokens)


def deserialize(data):
    """Read the tokens back in the same preorder with a moving index. O(n)."""
    tokens = data.split(",")
    index = [0]                            # one box all the calls share
    return read(tokens, index)


def read(tokens, index):
    """Build the subtree that starts at tokens[index[0]] and move past it."""
    token = tokens[index[0]]
    index[0] += 1
    if token == "#":
        return None
    node = TreeNode(int(token))
    node.left = read(tokens, index)        # the left subtree eats its tokens
    node.right = read(tokens, index)       # the right one starts after them
    return node


# --- try it ---
data = serialize(build_tree([1, 2, 3, None, None, 4, 5]))
print(data)                                # -> 1,2,#,#,3,4,#,#,5,#,#
print(tree_to_list(deserialize(data)))     # -> [1, 2, 3, None, None, 4, 5]
print(serialize(build_tree([-7, None, 12])))    # -> -7,#,12,#,#
```

```
print(serialize(None))                      # -> #
print(tree_to_list(deserialize("#")))       # -> []
```

56. BST Search, Insert, Delete

Delete with the In-Order Successor · BST · Tier 2 · python/056_bst_operations.py

Search, insert and delete values in a binary search tree while keeping its order (smaller to the left, larger to the right). Deleting 3 from [5, 3, 6, 2, 4, None, 7] → [5, 4, 6, 2, None, None, 7].

```python
from collections import deque      # popleft is O(1)


# --- helpers ---
# standard helpers TreeNode, build_tree, tree_to_list: printed once under Standard Helpers


# --- algorithm ---
def search(root, val):
    """Walk down one path: left if smaller, right if larger. O(h)."""
    node = root
    while node is not None and node.val != val:
        if val < node.val:
            node = node.left
        else:
            node = node.right
    return node                            # the node holding val, or None


def insert(root, val):
    """Descend to the empty spot where val belongs; hang a leaf there. O(h)."""
    if root is None:
        return TreeNode(val)               # the empty spot: a new leaf
    if val < root.val:
        root.left = insert(root.left, val)
    elif val > root.val:
        root.right = insert(root.right, val)
    return root                            # equal: already present


def delete(root, val):
    """Remove val; a node with two children takes its successor. O(h)."""
    if root is None:
        return None                        # val is not in the tree
    if val < root.val:
        root.left = delete(root.left, val)
    elif val > root.val:
        root.right = delete(root.right, val)
    else:
        if root.left is None:              # leaf, or right child only
            return root.right
        if root.right is None:             # left child only
            return root.left
        successor = root.right             # two children: leftmost node
        while successor.left is not None:  # of the right subtree
            successor = successor.left
        root.val = successor.val           # copy it up...
        root.right = delete(root.right, successor.val)  # ...and remove it
    return root
```

```
# --- try it ---
root = build_tree([5, 3, 6, 2, 4, None, 7])
print(tree_to_list(search(root, 3)))        # -> [3, 2, 4]
root = delete(root, 3)
print(tree_to_list(root))                   # -> [5, 4, 6, 2, None, None, 7]
print(search(root, 3))                      # -> None
root = insert(root, 1)
print(tree_to_list(root))                   # -> [5, 4, 6, 2, None, None, 7, 1]
```

57. BST Iterator

Controlled In-Order with a Stack · BST · Tier 2 · python/057_bst_iterator.py

Design an iterator over a binary search tree: next() returns the next smallest value and has_next() says whether any are left. Five calls to next() on [7, 3, 15, None, None, 9, 20] → 3, 7, 9, 15, 20.

```python
from collections import deque      # popleft is O(1)


# --- helpers ---
# standard helpers TreeNode, build_tree: printed once under Standard Helpers


# --- algorithm ---
class BSTIterator:
    """In-order iterator: the stack holds the nodes still waiting their turn."""

    def __init__(self, root):
        self.stack = []
        self.push_left(root)

    def push_left(self, node):
        """Push node and its left children; the smallest lands on top."""
        while node is not None:             # walk down the left spine
            self.stack.append(node)
            node = node.left

    def next(self):
        """Pop the smallest waiting node, then push the left spine of its
        right child. O(1) amortized, O(h) worst case."""
        node = self.stack.pop()             # the smallest value still waiting
        self.push_left(node.right)          # its successors live on the right
        return node.val

    def has_next(self):
        """True while some node is still waiting on the stack. O(1)."""
        return len(self.stack) > 0


def kth_smallest(root, k):
    """Call next() k times; the last value returned is the answer. O(h + k)."""
    iterator = BSTIterator(root)
    value = None
    for i in range(k):
        value = iterator.next()
    return value


# --- try it ---
iterator = BSTIterator(build_tree([7, 3, 15, None, None, 9, 20]))
print(iterator.next(), iterator.next(), iterator.has_next())   # -> 3 7 True
print(iterator.next(), iterator.next(), iterator.next())       # -> 9 15 20
print(iterator.has_next())                                     # -> False
root = build_tree([5, 3, 6, 2, 4, None, None, 1])
print(kth_smallest(root, 3))                                   # -> 3
```

58. Multi-Source BFS

Rotting Oranges · BFS · Tier 2 · python/058_multi_source_bfs.py

In a grid, 0 is empty, 1 is a fresh orange and 2 a rotten one. Every minute, each fresh orange next to a rotten one (up, down, left, right) rots. Return the minutes until no fresh orange is left, or -1 if one never rots: [[2, 1, 1, 1], [1, 0, 1, 1], [1, 1, 1, 2]] → 2.

```python
from collections import deque      # popleft is O(1)


# --- algorithm ---
DIRECTIONS = [(1, 0), (-1, 0), (0, 1), (0, -1)]


def oranges_rotting(grid):
    """BFS from every rotten orange at once; one layer per minute. O(R * C)."""
    rows = len(grid)
    cols = len(grid[0])
    queue = deque()
    fresh = 0
    for row in range(rows):
        for col in range(cols):
            if grid[row][col] == 2:
                queue.append((row, col))    # every source starts at minute 0
            elif grid[row][col] == 1:
                fresh += 1
    minutes = 0
    while queue and fresh > 0:
        for _ in range(len(queue)):         # exactly the oranges of this layer
            row, col = queue.popleft()
            for d_row, d_col in DIRECTIONS:
                next_row = row + d_row
                next_col = col + d_col
                inside = 0 <= next_row < rows and 0 <= next_col < cols
                if inside and grid[next_row][next_col] == 1:
                    grid[next_row][next_col] = 2    # rot it now: enqueued once
                    fresh -= 1
                    queue.append((next_row, next_col))
        minutes += 1
    if fresh > 0:
        return -1                           # walled off from every rotten one
    return minutes


# --- try it ---
print(oranges_rotting([[2, 1, 1, 1], [1, 0, 1, 1], [1, 1, 1, 2]]))  # -> 2
print(oranges_rotting([[2, 1, 1], [1, 1, 0], [0, 1, 1]]))           # -> 4
print(oranges_rotting([[2, 1, 1], [0, 1, 1], [1, 0, 1]]))           # -> -1
print(oranges_rotting([[0, 2]]))                                    # -> 0
```

59. Word Ladder

BFS on an Implicit Graph · BFS · Tier 2 · python/059_word_ladder.py

Turn begin_word into end_word one letter at a time, every word after the first taken from word_list. Return the number of words in the shortest such ladder, or 0 if there is none: “hit” to “cog” with [“hot”, “dot”, “dog”, “lot”, “log”, “cog”] → 5.

```python
from collections import deque      # popleft is O(1)


# --- algorithm ---
def ladder_length(begin_word, end_word, word_list):
    """BFS over words; neighbors differ in one letter. O(n * 26 * L^2)."""
    unvisited = set(word_list)          # dictionary words not reached yet
    if end_word not in unvisited:
        return 0
    if begin_word in unvisited:
        unvisited.remove(begin_word)    # the start counts as reached
    queue = deque([(begin_word, 1)])    # (word, words in the ladder so far)
    while queue:
        word, length = queue.popleft()
        if word == end_word:
            return length
        for i in range(len(word)):
            for letter in "abcdefghijklmnopqrstuvwxyz":
                candidate = word[:i] + letter + word[i + 1:]
                if candidate in unvisited:          # a word not reached yet
                    unvisited.remove(candidate)     # mark when enqueued
                    queue.append((candidate, length + 1))
    return 0


# --- try it ---
words = ["hot", "dot", "dog", "lot", "log", "cog"]
print(ladder_length("hit", "cog", words))                       # -> 5
print(ladder_length("hit", "cog", ["hot", "dot", "dog", "lot"]))  # -> 0
print(ladder_length("a", "c", ["a", "b", "c"]))                 # -> 2
```

60. Cycle Detection in a Directed Graph

Three Colors: White, Gray, Black · Cycle Detection · Tier 2 · python/060_directed_cycle_detection.py

Given n nodes 0..n-1 and directed edges [u, v] meaning u → v, report whether some path follows the arrows back to where it started: n = 5, [[0, 1], [0, 2], [1, 3], [2, 3], [2, 4], [4, 0]] → True.

```python
# --- algorithm ---
WHITE = 0                           # not visited yet
GRAY = 1                            # on the current DFS path
BLACK = 2                           # finished: no cycle is reachable from it


def has_cycle(n, edges):
    """DFS from every white node; an edge into a gray node closes a cycle.
    O(V + E) time."""
    graph = []
    for _ in range(n):
        graph.append([])
    for u, v in edges:
        graph[u].append(v)
    color = [WHITE] * n
    for node in range(n):
        if color[node] == WHITE and dfs(graph, node, color):
            return True
    return False


def dfs(graph, node, color):
    """True if a cycle is reachable from node; paints node black otherwise.
    O(V + E) over all calls."""
    color[node] = GRAY                  # node joins the current path
    for neighbor in graph[node]:
        if color[neighbor] == GRAY:
            return True                 # back edge to a node on the path
        if color[neighbor] == WHITE and dfs(graph, neighbor, color):
            return True
    color[node] = BLACK                 # every path out of node is cycle-free
    return False


# --- try it ---
edges = [[0, 1], [0, 2], [1, 3], [2, 3], [2, 4], [4, 0]]
print(has_cycle(5, edges))                              # -> True
print(has_cycle(4, [[0, 1], [0, 2], [1, 3], [2, 3]]))   # -> False
print(has_cycle(2, [[1, 1]]))                           # -> True
print(has_cycle(3, []))                                 # -> False
```

61. Topological Sort with DFS

Reverse Postorder · Topological Sort · Tier 2 · python/061_dfs_topological_sort.py

Given n nodes 0..n-1 and directed edges [u, v] meaning u must come before v, return an order of all the nodes that respects every edge, or [] if a cycle makes that impossible: n = 6, [[0, 3], [1, 3], [3, 4], [3, 2], [4, 5], [2, 5]] → [1, 0, 3, 2, 4, 5].

```
# --- algorithm ---
WHITE = 0                           # not visited yet
GRAY = 1                            # on the current DFS path
BLACK = 2                           # finished: placed after its descendants


def topological_sort(n, edges):
    """List nodes as they finish in a DFS, then reverse; [] on a cycle.
    O(V + E) time."""
    graph = []
    for _ in range(n):
        graph.append([])
    for u, v in edges:
        graph[u].append(v)
    color = [WHITE] * n
    finished = []                       # nodes in the order they finish
    for node in range(n):
        if color[node] == WHITE and not dfs(graph, node, color, finished):
            return []                   # a cycle: no order exists
    return finished[::-1]


def dfs(graph, node, color, finished):
    """Finish node after everything it points to; False if a cycle is found.
    O(V + E) over all calls."""
    color[node] = GRAY                  # node joins the current path
    for neighbor in graph[node]:
        if color[neighbor] == GRAY:
            return False                # back edge: a cycle
        if color[neighbor] == WHITE:
            if not dfs(graph, neighbor, color, finished):
                return False
    color[node] = BLACK
    finished.append(node)               # its descendants are all in already
    return True


# --- try it ---
edges = [[0, 3], [1, 3], [3, 4], [3, 2], [4, 5], [2, 5]]
print(topological_sort(6, edges))                       # -> [1, 0, 3, 2, 4, 5]
print(topological_sort(4, [[0, 1], [0, 2], [1, 3], [2, 3]]))  # -> [0, 2, 1, 3]
print(topological_sort(3, [[0, 1], [1, 2], [2, 0]]))    # -> []
print(topological_sort(2, []))                          # -> [1, 0]
```

62. Connected Components

Count and Label by Flood Fill · Connected Components · Tier 2 · python/062_connected_components.py

Given n nodes 0..n-1 and an undirected edge list, count the connected components and label every node with the number of its component: n = 7, [[0, 1], [1, 2], [3, 4], [3, 5]] → (3, [0, 0, 0, 1, 1, 1, 2]).

```python
# --- algorithm ---
def connected_components(n, edges):
    """Each unlabeled node floods its whole component with a new label.
    O(V + E) time."""
    graph = []
    for _ in range(n):
        graph.append([])
    for u, v in edges:
        graph[u].append(v)              # undirected: store both directions
        graph[v].append(u)
    label = [-1] * n                    # -1 means not reached yet
    count = 0
    for start in range(n):
        if label[start] == -1:          # not reached by any earlier flood
            flood(graph, start, count, label)
            count += 1
    return count, label


def flood(graph, start, mark, label):
    """Paint every node reachable from start with mark, using a stack.
    O(nodes + edges) of that component."""
    label[start] = mark
    stack = [start]
    while stack:
        node = stack.pop()
        for neighbor in graph[node]:
            if label[neighbor] == -1:
                label[neighbor] = mark  # label when pushed: pushed only once
                stack.append(neighbor)


# --- try it ---
edges = [[0, 1], [1, 2], [3, 4], [3, 5]]
print(connected_components(7, edges))       # -> (3, [0, 0, 0, 1, 1, 1, 2])
print(connected_components(3, []))          # -> (3, [0, 1, 2])
chain = [[3, 2], [1, 0], [2, 1]]
print(connected_components(4, chain))       # -> (1, [0, 0, 0, 0])
```

63. Word Search II

Trie-Guided Grid Backtracking · Tries · Tier 2 · python/063_word_search_ii.py

Given a grid of letters and a list of words, return every word that can be spelled along a path of adjacent cells (up, down, left, right), each cell used at most once per word: grid oaan / etae / ihkr / iflv with words oath, pea, eat, rain → [‘oath’, ‘eat’] (any order).

```python
# --- helpers ---
class TrieNode:
    def __init__(self):
        self.children = {}              # letter -> TrieNode
        self.word = None                # the whole word, at its last node


# --- algorithm ---
def build_trie(words):
    """Insert every word; its last node keeps the word. O(total letters)."""
    root = TrieNode()
    for word in words:
        node = root
        for ch in word:
            if ch not in node.children:
                node.children[ch] = TrieNode()
            node = node.children[ch]
        node.word = word
    return root


def dfs(board, row, col, parent, found):
    """Step onto (row, col) only if the trie goes on with its letter."""
    if row < 0 or row >= len(board) or col < 0 or col >= len(board[0]):
        return
    ch = board[row][col]
    if ch not in parent.children:
        return                          # no word goes this way (or a '#')
    node = parent.children[ch]
    if node.word is not None:
        found.append(node.word)
        node.word = None                # record each word only once
    board[row][col] = "#"               # mark: on the current path
    for d_row, d_col in [(1, 0), (-1, 0), (0, 1), (0, -1)]:
        dfs(board, row + d_row, col + d_col, node, found)
    board[row][col] = ch                # unmark on the way back
    if len(node.children) == 0:
        del parent.children[ch]         # prune: no word is left below


def find_words(board, words):
    """One trie, one DFS from every cell. O(m * n * 3^L), L the longest."""
    root = build_trie(words)
    found = []
    for row in range(len(board)):
        for col in range(len(board[0])):
            dfs(board, row, col, root, found)
    return found
```

```
# --- try it ---
board = [list("oaan"), list("etae"), list("ihkr"), list("iflv")]
print(find_words(board, ["oath", "pea", "eat", "rain"]))  # -> ['oath', 'eat']
print(find_words(board, ["oat", "oath", "eath"]))  # -> ['oat', 'oath', 'eath']
print(find_words([list("ab")], ["ab", "ba", "aba"]))      # -> ['ab', 'ba']
print(find_words([list("ab"), list("cd")], ["abcb"]))     # -> []
```

64. KMP String Matching

The Prefix Function Never Re-Reads the Text · Strings · Tier 2 · python/064_kmp.py

Return every index where the pattern starts in the text, overlapping matches included: text “abababc”, pattern “ababc” → [2].

```python
# --- algorithm ---
def build_lps(pattern):
    """lps[i] = length of the longest proper prefix of pattern[0..i] that is
    also its suffix; the search loop run on the pattern itself. O(m)."""
    lps = [0] * len(pattern)
    j = 0                                # length of the border being extended
    for i in range(1, len(pattern)):     # lps[0] = 0: a proper prefix is shorter
        while j > 0 and pattern[i] != pattern[j]:
            j = lps[j - 1]               # fall back to the next shorter border
        if pattern[i] == pattern[j]:
            j += 1
        lps[i] = j
    return lps


def kmp_search(text, pattern):
    """i never moves back; a mismatch slides the pattern by lps. O(n + m)."""
    if len(pattern) == 0:
        return []
    lps = build_lps(pattern)
    matches = []
    j = 0                                # pattern characters matched so far
    for i in range(len(text)):
        while j > 0 and text[i] != pattern[j]:
            j = lps[j - 1]               # slide the pattern; text[i] is retried
        if text[i] == pattern[j]:
            j += 1
        if j == len(pattern):
            matches.append(i - j + 1)    # the match ends at i
            j = lps[j - 1]               # keep the border: overlaps are found
    return matches


# --- try it ---
print(build_lps("ababc"))                # -> [0, 0, 1, 2, 0]
print(kmp_search("abababc", "ababc"))    # -> [2]
print(build_lps("aabaaab"))              # -> [0, 1, 0, 1, 2, 2, 3]
print(kmp_search("aaaaa", "aa"))         # -> [0, 1, 2, 3]
print(kmp_search("abc", "abd"))          # -> []
```

65. Longest Palindromic Substring

Expand Around Every Center · Strings · Tier 2 · python/065_longest_palindromic_substring.py

Return the longest contiguous substring that reads the same forwards and backwards (the leftmost one on a tie): “babad” → “bab”.

```python
# --- algorithm ---
def longest_palindrome(s):
    """Expand around all 2n - 1 centers (n chars, n - 1 gaps); keep the
    longest. O(n^2) time, O(1) space."""
    best_start = 0
    best_len = 0
    for center in range(2 * len(s) - 1):
        left = center // 2
        right = left + center % 2       # even center: a char; odd: a gap
        while left >= 0 and right < len(s) and s[left] == s[right]:
            left -= 1                   # ends match: grow one step each way
            right += 1
        length = right - left - 1       # the loop stepped one past each end
        if length > best_len:           # strictly longer: ties keep the first
            best_start = left + 1
            best_len = length
    return s[best_start:best_start + best_len]


# --- try it ---
print(longest_palindrome("babad"))          # -> bab
print(longest_palindrome("cbbd"))           # -> bb
print(longest_palindrome("abacdfgdcaba"))   # -> aba
print(longest_palindrome("a"))              # -> a
```

66. Encode and Decode Strings

Length-Prefix Framing · Strings · Tier 2 · python/066_encode_decode_strings.py

Pack a list of strings into one string and unpack it back to the same list; the strings may contain any character, '#' and digits included: ["ab#", "12", ""] →"3#ab#2#120#" → ["ab#", "12", ""].

```python
# --- algorithm ---
def encode(strs):
    """Prefix each string with its length and a '#'. O(total length)."""
    parts = []
    for s in strs:
        parts.append(str(len(s)) + "#" + s)
    return "".join(parts)               # join once; += in a loop may recopy


def decode(data):
    """Read digits up to '#', then take exactly that many characters.
    O(total length)."""
    strs = []
    i = 0
    while i < len(data):
        j = i
        while data[j] != "#":           # the length ends at the first '#'
            j += 1
        length = int(data[i:j])
        strs.append(data[j + 1:j + 1 + length])
        i = j + 1 + length              # jump over the payload unread
    return strs


# --- try it ---
print(encode(["ab#", "12", ""]))                  # -> 3#ab#2#120#
print(decode("3#ab#2#120#"))                      # -> ['ab#', '12', '']
print(decode(encode(["hello world", "#4#"])))     # -> ['hello world', '#4#']
print(decode(encode([])), decode(encode([""])))   # -> [] ['']
```

67. Fibonacci

Recursion, Memo, Iteration, Fast Doubling · Math · Tier 2 · python/067_fibonacci.py

F(0) = 0, F(1) = 1, and every later term is the sum of the two before it: 0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55. Compute F(n): n = 10 → 55.

```python
# --- algorithm ---
def fib_naive(n):
    """Two calls per call: 2F(n + 1) - 1 calls, growing like 1.618^n."""
    if n < 2:
        return n
    return fib_naive(n - 1) + fib_naive(n - 2)


def fib_memo(n, memo):
    """The same recursion, but each F(k) is solved once. O(n) time and space."""
    if n < 2:
        return n
    if n in memo:
        return memo[n]                  # solved before: skip the whole subtree
    memo[n] = fib_memo(n - 1, memo) + fib_memo(n - 2, memo)
    return memo[n]


def fib_iter(n):
    """Slide a window of two neighbors n steps. O(n) time, O(1) space."""
    prev = 0
    curr = 1
    for i in range(n):                  # prev = F(i), curr = F(i + 1)
        nxt = prev + curr
        prev = curr
        curr = nxt
    return prev


def fib_pair(n):
    """Return (F(n), F(n + 1)) from the pair at n // 2. O(log n) calls."""
    if n == 0:
        return (0, 1)
    a, b = fib_pair(n // 2)             # a = F(k), b = F(k + 1) with k = n // 2
    even = a * (2 * b - a)              # F(2k)
    odd = a * a + b * b                 # F(2k + 1)
    if n % 2 == 0:
        return (even, odd)
    return (odd, even + odd)            # n = 2k + 1: one step further


def fib_fast(n):
    """Fast doubling: F(n) is the first value of the pair. O(log n) steps."""
    pair = fib_pair(n)
    return pair[0]


# --- try it ---
print(fib_naive(10))                    # -> 55
```

```
print(fib_memo(50, {}))                 # -> 12586269025
print(fib_iter(90))                     # -> 2880067194370816120
print(fib_fast(100))                    # -> 354224848179261915075
```

68. Single Number

XOR Cancels Pairs · Bit Manipulation · Tier 2 · python/068_single_number_xor.py

Every element of a non-empty array appears twice except one, which appears once; find it in O(n) time and O(1) extra space: [4, 1, 2, 1, 2] → 4.

```python
# --- algorithm ---
def single_number(nums):
    """XOR everything: each pair cancels to 0, the single survives. O(n)."""
    result = 0
    for value in nums:
        result ^= value                 # x ^ x = 0 and x ^ 0 = x
    return result


def missing_number(nums):
    """nums holds 0..n with one value missing; XOR indices and values. O(n)."""
    result = len(nums)                  # index n has no slot of its own
    for i in range(len(nums)):
        result ^= i                     # value v cancels against index v
        result ^= nums[i]
    return result


# --- try it ---
print(single_number([4, 1, 2, 1, 2]))     # -> 4
print(single_number([-3, 7, 7]))          # -> -3
print(missing_number([3, 0, 1]))          # -> 2
print(missing_number([0, 1]))             # -> 2
```

69. Insert Delete GetRandom O(1)

Array + Index Map, Swap with Last · Design · Tier 2 · python/069_randomized_set.py

Design a set with insert, remove and get_random in average O(1) time each, where get_random returns every stored value with equal probability: insert 10, 20, 30, then remove 10 → values [30, 20].

```python
import random                       # randint(a, b) is uniform on a..b inclusive


# --- algorithm ---
class RandomizedSet:
    """Packed list for random picks, map value -> index for lookups. O(1)."""

    def __init__(self):
        self.values = []            # packed from index 0, no holes
        self.index_of = {}          # value -> its slot in values

    def insert(self, val):
        """Append at the end and remember the slot. O(1)."""
        if val in self.index_of:
            return False
        self.index_of[val] = len(self.values)
        self.values.append(val)
        return True

    def remove(self, val):
        """Move the last value into val's slot, then pop the end. O(1)."""
        if val not in self.index_of:
            return False
        i = self.index_of[val]
        last = self.values[-1]
        self.values[i] = last       # the last value fills the hole
        self.index_of[last] = i     # and learns its new slot
        self.values.pop()           # drop the tail slot
        del self.index_of[val]      # only now: last may be val
        return True

    def get_random(self):
        """Pick a uniform random slot of the packed list. O(1)."""
        i = random.randint(0, len(self.values) - 1)
        return self.values[i]


# --- try it ---
s = RandomizedSet()
s.insert(10)
s.insert(20)
s.insert(30)
print(s.insert(20), s.remove(10), s.remove(10))   # -> False True False
print(s.values, s.index_of)                       # -> [30, 20] {20: 1, 30: 0}
print(s.get_random() in (20, 30))                 # -> True
print(s.remove(20), s.values, s.index_of)         # -> True [30] {30: 0}
```

70. Time-Based Key-Value Store

Per-Key Timestamp Lists + Binary Search · Design · Tier 2 · python/070_time_based_kv_store.py

Keep every version of a key: set(key, value, t) records value at time t (the times of each key only grow), and get(key, t) returns the value set at the largest time <= t, or "" if none: set status = draft at 1, review at 4, live at 9; get status at 5 → "review".

```python
# --- algorithm ---
class TimeMap:
    """Per key, parallel lists of times and values. set O(1), get O(log n)."""

    def __init__(self):
        self.times = {}            # key -> increasing list of set times
        self.values = {}           # key -> the values, in the same order

    def set(self, key, value, timestamp):
        """Append the version to both lists of the key. O(1) amortized."""
        if key not in self.times:
            self.times[key] = []
            self.values[key] = []
        self.times[key].append(timestamp)   # times only grow: stays sorted
        self.values[key].append(value)

    def get(self, key, timestamp):
        """First time > timestamp, minus one: the last time <= it. O(log n)."""
        if key not in self.times:
            return ""
        times = self.times[key]
        left = 0
        right = len(times)
        while left < right:             # find the first time > timestamp
            mid = (left + right) // 2
            if times[mid] <= timestamp:
                left = mid + 1          # mid is not after timestamp
            else:
                right = mid             # mid is after: maybe the first
        if left == 0:
            return ""                   # every version is newer
        return self.values[key][left - 1]


# --- try it ---
store = TimeMap()
store.set("status", "draft", 1)
store.set("status", "review", 4)
store.set("status", "live", 9)
print(store.get("status", 5), store.get("status", 9))    # -> review live
print(store.get("status", 4), store.get("status", 100))  # -> review live
print(repr(store.get("status", 0)), repr(store.get("owner", 5)))   # -> '' ''
```

Tier 3 · Good to Know

The last thirty round out the set: second implementations of ideas you already own (open addressing, heap sort, Prim), the string toolkit, number theory, geometry, bit tricks and the designs that turn up in senior loops. Read each one once, keep its picture in mind, and come back to it when a problem calls for it.

71. Hash Map with Open Addressing

Linear Probing and Tombstones · Hash Map Internals · Tier 3 · python/071_hash_map_open_addressing.py

Design a hash map without a built-in dict that stores every pair in one array of slots: put(key, value), get(key) (-1 if missing) and remove(key). With 8 slots, keys 3 and 11 both start at slot 3, so 11 moves on to slot 4: put(3, 30), put(11, 110), get(11) → 110.

```
# --- algorithm ---
EMPTY = None                # never used: a search can stop here
DELETED = "DELETED"         # tombstone: used once, a search must go on


class ProbingHashMap:
    def __init__(self, capacity=8):
        self.capacity = capacity
        self.size = 0                       # live keys
        self.tombstones = 0                 # DELETED slots
        self.slots = [EMPTY] * capacity     # EMPTY, DELETED or [key, value]

    def find(self, key):
        """Index of key's slot or -1: probe until the key or an EMPTY slot.
        Expected O(1)."""
        index = hash(key) % self.capacity
        while self.slots[index] is not EMPTY:
            slot = self.slots[index]
            if slot != DELETED and slot[0] == key:
                return index
            index = (index + 1) % self.capacity     # taken: try the next slot
        return -1

    def put(self, key, value):
        """Overwrite if present, else fill the first free slot on the path.
        Expected O(1), amortized over the resizes."""
        index = self.find(key)
        if index != -1:
            self.slots[index][1] = value            # already stored: overwrite
            return
        index = hash(key) % self.capacity
        while self.slots[index] is not EMPTY and self.slots[index] != DELETED:
            index = (index + 1) % self.capacity     # skip live pairs
        if self.slots[index] == DELETED:
            self.tombstones -= 1                    # reuse the tombstone
        self.slots[index] = [key, value]
        self.size += 1
        if (self.size + self.tombstones) / self.capacity > 0.5:
            self.resize()

    def get(self, key):
        """Value of key or -1. Expected O(1) while the table is half empty."""
        index = self.find(key)
        if index == -1:
            return -1
        return self.slots[index][1]

    def remove(self, key):
```

```
        """Leave a tombstone so that searches step over it. Expected O(1)."""
        index = self.find(key)
        if index != -1:
            self.slots[index] = DELETED
            self.size -= 1
            self.tombstones += 1

    def resize(self):
        """Double the array and re-put the live pairs only. O(n)."""
        old_slots = self.slots
        self.capacity = 2 * self.capacity
        self.slots = [EMPTY] * self.capacity
        self.size = 0
        self.tombstones = 0
        for slot in old_slots:
            if slot is not EMPTY and slot != DELETED:
                self.put(slot[0], slot[1])          # tombstones are dropped


# --- try it ---
table = ProbingHashMap()
table.put(3, 30)
table.put(11, 110)                  # home slot 3 is taken: lands in slot 4
table.put(19, 190)                  # slots 3 and 4 are taken: slot 5
table.put(4, 40)                    # home slot 4 is inside the run: slot 6
table.remove(11)                    # slot 4 becomes a tombstone
print(table.get(19), table.get(11), table.tombstones)   # -> 190 -1 1
table.put(27, 270)                  # reuses the tombstone in slot 4
print(table.slots[4], table.tombstones)                 # -> [27, 270] 0
table.put(12, 120)                  # 5 of 8 slots used: grow to 16
print(table.capacity, table.size, table.get(12))        # -> 16 5 120
```

72. Boyer-Moore Majority Vote

Different Votes Cancel in Pairs · Arrays & Hashing · Tier 3 · python/072_boyer_moore_majority_vote.py

Given an array, return the value that appears more than n / 2 times, or None if no value does, in O(n) time and O(1) extra space: [2, 2, 1, 1, 1, 2, 2] → 2.

```python
# --- algorithm ---
def majority_candidate(nums):
    """Pair off different values; only a majority must survive. O(n), O(1)."""
    candidate = None
    count = 0                       # candidate votes not yet cancelled
    for x in nums:
        if count == 0:
            candidate = x           # nobody standing: x takes over
            count = 1
        elif x == candidate:
            count += 1              # one more vote for the candidate
        else:
            count -= 1              # x cancels one candidate vote
    return candidate


def majority_element(nums):
    """Second pass: the candidate wins only with more than n / 2 votes."""
    candidate = majority_candidate(nums)
    votes = 0
    for x in nums:
        if x == candidate:
            votes += 1
    if votes * 2 > len(nums):
        return candidate
    return None                     # no value holds a strict majority


# --- try it ---
print(majority_element([2, 2, 1, 1, 1, 2, 2]))  # -> 2
print(majority_element([3, 2, 3]))              # -> 3
print(majority_candidate([1, 2, 3]))            # -> 3
print(majority_element([1, 2, 3]))              # -> None
print(majority_element([1, 2]))                 # -> None
```

73. Median of K Sorted Arrays

Binary Search on the Value · Binary Search · Tier 3 · python/073_median_k_sorted_arrays.py

Given k sorted arrays of integers holding N >= 1 values in total, return the median of all of them (the average of the two middle values when N is even): [[1, 4, 9], [2, 3, 7, 10], [5, 8]] → 5.

```python
import math                          # math.inf seeds the running min and max


# --- algorithm ---
def count_at_most(arr, value):
    """Upper bound (#9): how many values of sorted arr are <= value.
    O(log L)."""
    lo = 0
    hi = len(arr)
    while lo < hi:
        mid = (lo + hi) // 2
        if arr[mid] <= value:
            lo = mid + 1
        else:
            hi = mid
    return lo


def kth_smallest(arrays, rank):
    """Smallest value v with at least rank values <= v, by binary search on
    v from the smallest to the largest value. O(k log L log R)."""
    lo = math.inf
    hi = -math.inf
    for arr in arrays:
        if len(arr) > 0:
            lo = min(lo, arr[0])         # smallest first element
            hi = max(hi, arr[-1])        # largest last element
    while lo < hi:
        mid = (lo + hi) // 2
        count = 0
        for arr in arrays:
            count += count_at_most(arr, mid)
        if count >= rank:
            hi = mid                     # mid already covers rank values
        else:
            lo = mid + 1                 # too few values are <= mid
    return lo


def median_k_sorted(arrays):
    """One order statistic for odd N, the average of two for even N.
    O(k log L log R)."""
    total = 0
    for arr in arrays:
        total += len(arr)
    if total % 2 == 1:
        return kth_smallest(arrays, total // 2 + 1)
    low = kth_smallest(arrays, total // 2)
    high = kth_smallest(arrays, total // 2 + 1)
```

```
    return (low + high) / 2


# --- try it ---
arrays = [[1, 4, 9], [2, 3, 7, 10], [5, 8]]
print(median_k_sorted(arrays))                    # -> 5
print(kth_smallest(arrays, 6))                    # -> 7
print(median_k_sorted([[1, 2], [3, 4]]))          # -> 2.5
print(median_k_sorted([[-5, -1], [], [-3]]))      # -> -3
```

74. Counting Sort

Count, Prefix-Sum, Place · Sorting · Tier 3 · python/074_counting_sort.py

Sort integers that lie in a small range [low, high], negatives included, without comparing them: [2, -1, 3, 2, 0, -1] → [-1, -1, 0, 2, 2, 3].

```python
# --- algorithm ---
def counting_sort(nums):
    """Count each value, turn the counts into end positions, place each value.
    O(n + k) time and space for k = high - low + 1 possible values."""
    if len(nums) == 0:
        return []
    low = min(nums)
    high = max(nums)
    count = [0] * (high - low + 1)          # count[v - low] is for value v
    for value in nums:
        count[value - low] += 1
    for slot in range(1, len(count)):       # now: how many values are <= it
        count[slot] += count[slot - 1]
    result = [0] * len(nums)
    for i in range(len(nums) - 1, -1, -1):  # right to left keeps it stable
        value = nums[i]
        count[value - low] -= 1             # the last free seat
        result[count[value - low]] = value
    return result


# --- try it ---
print(counting_sort([2, -1, 3, 2, 0, -1]))   # -> [-1, -1, 0, 2, 2, 3]
print(counting_sort([5, 3, 5, 1]))           # -> [1, 3, 5, 5]
print(counting_sort([4]))                    # -> [4]
print(counting_sort([]))                     # -> []
```

75. Heap Sort

Build a Max-Heap, Swap the Max to the End · Sorting · Tier 3 · python/075_heap_sort.py

Sort an array of integers in place, with O(1) extra space and an O(n log n) bound even in the worst case: [4, 10, 3, 5, 1] → [1, 3, 4, 5, 10].

```python
# --- algorithm ---
def sift_down(nums, i, n):
    """Max-heap sift-down: swap nums[i] with its larger child until no child
    is larger; only nums[0..n-1] is the heap. O(log n)."""
    while True:
        largest = i
        left = 2 * i + 1
        right = 2 * i + 2
        if left < n and nums[left] > nums[largest]:
            largest = left
        if right < n and nums[right] > nums[largest]:
            largest = right
        if largest == i:                # no child is larger: done
            return
        nums[i], nums[largest] = nums[largest], nums[i]
        i = largest


def heap_sort(nums):
    """Build a max-heap, then move the max to the end n - 1 times.
    O(n log n) time, O(1) extra space."""
    n = len(nums)
    for i in range(n // 2 - 1, -1, -1):          # build: last parent to root
        sift_down(nums, i, n)
    for end in range(n - 1, 0, -1):
        nums[0], nums[end] = nums[end], nums[0]  # the max joins the sorted tail
        sift_down(nums, 0, end)                  # the heap is nums[0..end-1]
    return nums


# --- try it ---
print(heap_sort([4, 10, 3, 5, 1]))    # -> [1, 3, 4, 5, 10]
print(heap_sort([2, -7, 2, 0, -7]))   # -> [-7, -7, 0, 2, 2]
print(heap_sort([9]))                 # -> [9]
print(heap_sort([]))                  # -> []
```

76. Circular Queue

Ring Buffer with Head and Count · Queues · Tier 3 · python/076_circular_queue.py

Design a FIFO queue of fixed capacity on one array whose end wraps around to its start: enqueue and dequeue return True or False, front and rear return -1 when empty. Capacity 3: enqueue 1, 2, 3, then enqueue 4 → False (full).

```python
# --- algorithm ---
class CircularQueue:
    def __init__(self, capacity):
        self.items = [0] * capacity
        self.capacity = capacity
        self.head = 0                   # index of the front value
        self.count = 0                  # how many values are stored

    def enqueue(self, value):
        """Write at tail = (head + count) mod capacity. O(1)."""
        if self.is_full():
            return False
        tail = (self.head + self.count) % self.capacity    # wraps to 0
        self.items[tail] = value
        self.count += 1
        return True

    def dequeue(self):
        """Drop the front value by moving head one step, wrapping. O(1)."""
        if self.is_empty():
            return False
        self.head = (self.head + 1) % self.capacity    # old value stays
        self.count -= 1
        return True

    def front(self):
        """The oldest value, or -1 if empty. O(1)."""
        if self.is_empty():
            return -1
        return self.items[self.head]

    def rear(self):
        """The newest value, at (head + count - 1) mod capacity. O(1)."""
        if self.is_empty():
            return -1
        return self.items[(self.head + self.count - 1) % self.capacity]

    def is_empty(self):
        """No values stored. O(1)."""
        return self.count == 0

    def is_full(self):
        """Every slot holds a value. O(1)."""
        return self.count == self.capacity


# --- try it ---
queue = CircularQueue(3)
```

```
queue.enqueue(1)
queue.enqueue(2)
queue.enqueue(3)
print(queue.enqueue(4), queue.rear())               # -> False 3
print(queue.dequeue(), queue.enqueue(4))            # -> True True
print(queue.front(), queue.rear(), queue.items)     # -> 2 4 [4, 2, 3]
print(queue.is_full(), queue.head, queue.count)     # -> True 1 3
```

77. Candy

Two Passes: Left, Then Right · Greedy · Tier 3 · python/077_candy.py

Children stand in a line with ratings. Each gets at least one candy, and a child rated higher than a neighbor gets more candies than that neighbor. Return the fewest candies in total: [1, 0, 2, 5, 4, 3, 1] → 15.

```python
# --- algorithm ---
def candy(ratings):
    """Left pass for left neighbors, right pass for right ones. O(n) time."""
    n = len(ratings)
    candies = [1] * n               # everyone gets at least one
    for i in range(1, n):           # left to right
        if ratings[i] > ratings[i - 1]:     # beat a lower left neighbor
            candies[i] = candies[i - 1] + 1
    for i in range(n - 2, -1, -1):  # right to left
        if ratings[i] > ratings[i + 1]:     # also beat a lower right neighbor
            candies[i] = max(candies[i], candies[i + 1] + 1)
    return sum(candies)


# --- try it ---
print(candy([1, 0, 2, 5, 4, 3, 1]))     # -> 15
print(candy([1, 0, 2]))                 # -> 5
print(candy([1, 2, 2]))                 # -> 4
print(candy([1, 2, 3, 1]))              # -> 7
```

78. Hand of Straights

Start Every Group at the Smallest Card · Greedy · Tier 3 · python/078_hand_of_straights.py

Can the cards be split into groups of group_size consecutive values, every card used exactly once? [1, 2, 3, 6, 2, 3, 4, 7, 8], group_size 3 → True (as [1, 2, 3], [2, 3, 4] and [6, 7, 8]).

```python
# --- algorithm ---
def can_form_straights(hand, group_size):
    """The smallest card left must start a group. O(n log n) for the sort."""
    if len(hand) % group_size != 0:
        return False
    count = {}
    for card in hand:
        count[card] = count.get(card, 0) + 1
    for card in sorted(count):          # distinct cards, smallest first
        starts = count[card]            # groups that must start at this card
        if starts > 0:
            for next_card in range(card, card + group_size):
                if count.get(next_card, 0) < starts:
                    return False        # a group would have a gap
                count[next_card] -= starts
    return True


# --- try it ---
print(can_form_straights([1, 2, 3, 6, 2, 3, 4, 7, 8], 3))   # -> True
print(can_form_straights([1, 2, 4, 5, 6, 7], 3))            # -> False
print(can_form_straights([1, 2, 3, 4, 5], 4))               # -> False
print(can_form_straights([2, 1, 3], 3))                     # -> True
```

79. N-Queens

Columns and Diagonals as Sets · Backtracking · Tier 3 · python/079_n_queens.py

Place n queens on an n x n board so that no two share a row, a column or a diagonal, and return every such board as a list of row strings: n = 4 → two boards, [‘.Q..’, ‘…Q’, ‘Q…’, ‘..Q.’] and its mirror image.

```python
# --- helpers ---
def draw(queens):
    """queens[r] is the column of row r's queen; spell each row. O(n^2)."""
    n = len(queens)
    board = []
    for col in queens:
        board.append("." * col + "Q" + "." * (n - 1 - col))
    return board


# --- algorithm ---
def backtrack(n, row, queens, cols, diags, anti_diags, boards):
    """Try each safe column in this row, then fill the rows below it."""
    if row == n:
        boards.append(draw(queens))     # every row holds a queen
        return
    for col in range(n):
        if col in cols or (row - col) in diags or (row + col) in anti_diags:
            continue                    # attacked: a column or a diagonal
        queens.append(col)              # place the queen
        cols.add(col)
        diags.add(row - col)
        anti_diags.add(row + col)
        backtrack(n, row + 1, queens, cols, diags, anti_diags, boards)
        queens.pop()                    # remove it, undoing all four marks
        cols.remove(col)
        diags.remove(row - col)
        anti_diags.remove(row + col)


def solve_n_queens(n):
    """One queen per row; three sets make each safety check O(1)."""
    boards = []
    backtrack(n, 0, [], set(), set(), set(), boards)
    return boards


# --- try it ---
boards = solve_n_queens(4)
print(boards[0])                    # -> ['.Q..', '...Q', 'Q...', '..Q.']
print(boards[1])                    # -> ['..Q.', 'Q...', '...Q', '.Q..']
print(len(solve_n_queens(8)))       # -> 92
print(solve_n_queens(3))            # -> []
print(solve_n_queens(1))            # -> [['Q']]
```

80. Bipartite Check

BFS Two-Coloring · BFS · Tier 3 · python/080_bipartite_check.py

Given an undirected graph as an adjacency list (graph[u] lists the neighbors of u), decide whether its nodes split into two sides with every edge running between the sides. A square 0-1-2-3 plus a separate triangle 4-5-6: [[1, 3], [0, 2], [1, 3], [0, 2], [5, 6], [4, 6], [4, 5]] → False.

```python
from collections import deque      # popleft is O(1)


# --- algorithm ---
def is_bipartite(graph):
    """BFS each component, giving every neighbor the opposite color; an edge
    inside one color means an odd cycle. O(V + E) time."""
    n = len(graph)
    color = [-1] * n                    # -1 uncolored, else side 0 or 1
    for start in range(n):
        if color[start] != -1:
            continue                    # colored by an earlier search
        color[start] = 0                # a new component: either side works
        queue = deque([start])
        while queue:
            node = queue.popleft()
            for neighbor in graph[node]:
                if color[neighbor] == -1:
                    color[neighbor] = 1 - color[node]   # the other side
                    queue.append(neighbor)
                elif color[neighbor] == color[node]:
                    return False        # an edge inside one side
    return True


# --- try it ---
graph = [[1, 3], [0, 2], [1, 3], [0, 2], [5, 6], [4, 6], [4, 5]]
print(is_bipartite(graph))                                      # -> False
print(is_bipartite([[1, 3], [0, 2], [1, 3], [0, 2]]))       # -> True
print(is_bipartite([[1, 2, 3], [0, 2], [0, 1, 3], [0, 2]]))  # -> False
print(is_bipartite([[], [2], [1]]))                         # -> True
```

81. Cycle Detection in an Undirected Graph

DFS That Skips the Parent Edge · Cycle Detection · Tier 3 · python/081_undirected_cycle_detection.py

Given n nodes and an undirected edge list (no repeated edges), decide whether the graph has a cycle; it may be disconnected: n = 6, edges 0-1, 2-3, 3-4, 4-5, 5-2 → True (2, 3, 4, 5 form a square).

```python
# --- algorithm ---
def has_cycle(n, edges):
    """DFS from every unvisited node; a visited neighbor that is not the
    parent closes a cycle. O(V + E) time, O(V + E) space."""
    graph = []
    for _ in range(n):
        graph.append([])
    for u, v in edges:
        graph[u].append(v)
        graph[v].append(u)              # undirected: store both directions
    visited = [False] * n
    for start in range(n):              # a cycle can be in any component
        if visited[start]:
            continue
        visited[start] = True
        stack = []
        stack.append((start, -1))       # (node, the node we came from)
        while stack:
            node, parent = stack.pop()
            for neighbor in graph[node]:
                if not visited[neighbor]:
                    visited[neighbor] = True    # mark when discovered
                    stack.append((neighbor, node))
                elif neighbor != parent:
                    return True         # reached again, not via parent
    return False


# --- try it ---
print(has_cycle(6, [[0, 1], [2, 3], [3, 4], [4, 5], [5, 2]]))   # -> True
print(has_cycle(5, [[0, 1], [0, 2], [0, 3], [1, 4]]))           # -> False
print(has_cycle(4, [[0, 1], [2, 3]]))                           # -> False
print(has_cycle(3, [[0, 1], [1, 2], [2, 0]]))                   # -> True
```

82. Bellman-Ford

Relax Every Edge n - 1 Times · Shortest Path · Tier 3 · python/082_bellman_ford.py

Given n nodes, directed edges [u, v, w] whose weights may be negative, and a source, return the shortest distance to every node (inf if unreachable), or None if a negative cycle is reachable: n = 4, source 0, edges [[2, 3, 1], [1, 2, -3], [0, 1, 4], [0, 2, 2]] → [0, 4, 1, 2].

```python
import math                             # math.inf: no path found yet


# --- algorithm ---
def bellman_ford(n, edges, source):
    """Relax every edge, round after round, at most n - 1 rounds; a later
    improvement means a negative cycle. O(V * E) time, O(V) space."""
    dist = [math.inf] * n
    dist[source] = 0
    for _ in range(n - 1):              # a shortest path has <= n - 1 edges
        changed = False
        for u, v, w in edges:
            if dist[u] + w < dist[v]:   # inf + w stays inf: no false relax
                dist[v] = dist[u] + w   # a cheaper way into v, through u
                changed = True
        if not changed:
            break                       # quiet round: all distances final
    for u, v, w in edges:
        if dist[u] + w < dist[v]:       # still improving after n - 1 rounds
            return None                 # a negative cycle is reachable
    return dist


# --- try it ---
edges = [[2, 3, 1], [1, 2, -3], [0, 1, 4], [0, 2, 2]]
print(bellman_ford(4, edges, 0))                    # -> [0, 4, 1, 2]
print(bellman_ford(3, [[0, 1, 5]], 0))              # -> [0, 5, inf]
reachable_cycle = [[0, 1, 1], [1, 2, -1], [2, 1, -1]]
print(bellman_ford(3, reachable_cycle, 0))          # -> None
unreachable_cycle = [[0, 1, 2], [2, 3, -1], [3, 2, -1]]
print(bellman_ford(4, unreachable_cycle, 0))        # -> [0, 2, inf, inf]
```

83. Kruskal's Minimum Spanning Tree

Sort Edges, Let Union-Find Reject Cycles · MST · Tier 3 · python/083_kruskal_mst.py

Given n nodes and undirected edges [u, v, w], return the total weight and the edges of a minimum spanning tree, or None if the graph is disconnected: n = 5, edges 0-1 (4), 0-2 (1), 1-2 (2), 1-3 (5), 2-3 (8), 3-4 (3) → (11, [[0, 2, 1], [1, 2, 2], [3, 4, 3], [1, 3, 5]]).

```python
# --- algorithm ---
def find(parent, x):
    """Root of x; repoint the path straight at it. Near O(1) amortized."""
    if parent[x] != x:
        parent[x] = find(parent, parent[x])     # path compression
    return parent[x]


def union(parent, size, a, b):
    """Merge the components of a and b; False if they are already one."""
    root_a = find(parent, a)
    root_b = find(parent, b)
    if root_a == root_b:
        return False                    # already one component
    if size[root_a] < size[root_b]:
        root_a, root_b = root_b, root_a     # hang the smaller tree below
    parent[root_b] = root_a
    size[root_a] += size[root_b]
    return True


def kruskal(n, edges):
    """Take edges cheapest first; keep one only if it joins two components.
    O(E log E) for the sort."""
    order = []
    for u, v, w in edges:
        order.append((w, u, v))         # weight first: sort by weight
    order.sort()                        # cheapest edge first
    parent = list(range(n))             # every node is its own root
    size = [1] * n
    total = 0
    tree = []
    for w, u, v in order:
        if union(parent, size, u, v):   # two components: edge is safe
            total += w
            tree.append([u, v, w])
            if len(tree) == n - 1:
                break                   # n - 1 edges span all n nodes
    if len(tree) < n - 1:
        return None                     # the graph is disconnected
    return total, tree


# --- try it ---
edges = [[0, 1, 4], [0, 2, 1], [1, 2, 2], [1, 3, 5], [2, 3, 8], [3, 4, 3]]
total, tree = kruskal(5, edges)
print(total)                                # -> 11
print(tree)                 # -> [[0, 2, 1], [1, 2, 2], [3, 4, 3], [1, 3, 5]]
```

```
print(kruskal(4, [[0, 1, 1], [2, 3, 1]]))   # -> None
print(kruskal(1, []))                       # -> (0, [])
```

84. Prim's Minimum Spanning Tree

Grow One Tree with a Min-Heap · MST · Tier 3 · python/084_prim_mst.py

Given n nodes and undirected edges [u, v, w], return the total weight of a minimum spanning tree, or None if the graph is disconnected: n = 5, edges 0-1 (4), 0-2 (1), 1-2 (2), 1-3 (5), 2-3 (8), 3-4 (3) → 11.

```python
import heapq            # heappush / heappop keep the smallest at index 0


# --- algorithm ---
def prim(n, edges):
    """Grow one tree from node 0, always adding the cheapest edge that
    leaves it. O(E log V) time, O(V + E) space."""
    graph = []
    for _ in range(n):
        graph.append([])
    for u, v, w in edges:
        graph[u].append((w, v))
        graph[v].append((w, u))         # undirected: store both directions
    in_tree = [False] * n
    heap = []
    heapq.heappush(heap, (0, 0))        # (cost to attach, node): start at 0
    total = 0
    count = 0
    while heap and count < n:
        cost, node = heapq.heappop(heap)
        if in_tree[node]:
            continue                    # stale: joined by a cheaper edge
        in_tree[node] = True            # mark when popped, not when pushed
        total += cost
        count += 1
        for weight, neighbor in graph[node]:
            if not in_tree[neighbor]:
                heapq.heappush(heap, (weight, neighbor))
    if count < n:
        return None                     # some node was never reached
    return total


# --- try it ---
edges = [[0, 1, 4], [0, 2, 1], [1, 2, 2], [1, 3, 5], [2, 3, 8], [3, 4, 3]]
print(prim(5, edges))                               # -> 11
print(prim(4, [[0, 1, 1], [2, 3, 1]]))              # -> None
print(prim(3, [[0, 1, 5], [1, 2, -2], [0, 2, 1]]))  # -> -1
print(prim(1, []))                                  # -> 0
```

85. Rabin-Karp

Rolling Hash Pattern Matching · Strings · Tier 3 · python/085_rabin_karp.py

Return every index where the pattern starts in the text, comparing window hashes first and confirming each hash hit character by character: text “abracadabra”, pattern “abra” → [0, 7].

```python
# --- algorithm ---
BASE = 256                              # one digit per character code
MOD = 1000000007                        # a large prime keeps collisions rare


def poly_hash(s):
    """s read as a number in base BASE, kept mod MOD. O(len(s))."""
    value = 0
    for ch in s:
        value = (value * BASE + ord(ch)) % MOD  # shift left, add a digit
    return value


def rabin_karp(text, pattern):
    """Roll one window hash across the text; compare characters only on a
    hash hit. Expected O(n + m) time when matches are few."""
    n = len(text)
    m = len(pattern)
    if m == 0 or m > n:
        return []
    high = pow(BASE, m - 1, MOD)        # leading digit weight: BASE^(m-1) % MOD
    pattern_hash = poly_hash(pattern)
    window_hash = poly_hash(text[0:m])
    matches = []
    for start in range(n - m + 1):
        if window_hash == pattern_hash and text[start:start + m] == pattern:
            matches.append(start)       # a hash hit, confirmed
        if start + m < n:               # roll the window one step right
            window_hash = (window_hash - ord(text[start]) * high) % MOD
            window_hash = (window_hash * BASE + ord(text[start + m])) % MOD
    return matches


# --- try it ---
print(rabin_karp("abracadabra", "abra"))                # -> [0, 7]
print(rabin_karp("aaaaa", "aa"))                        # -> [0, 1, 2, 3]
print(poly_hash("neonatus") == poly_hash("plumular"))   # -> True
print(rabin_karp("neonatus", "plumular"))               # -> []
print(rabin_karp("abc", "abcd"))                        # -> []
```

86. Longest Common Prefix

Vertical Scan · Strings · Tier 3 · python/086_longest_common_prefix.py

Return the longest string that is a prefix of every string in the list, or “” if they share none: [“flower”, “flow”, “flight”] → “fl”.

```python
# --- algorithm ---
def longest_common_prefix(strs):
    """Scan column by column; stop at the first mismatch or the end of the
    shortest string. O(n * (L + 1)) for n strings and answer length L."""
    if len(strs) == 0:
        return ""
    first = strs[0]
    for col in range(len(first)):
        ch = first[col]
        for word in strs:
            if col == len(word) or word[col] != ch:   # word ended, or mismatch
                return first[0:col]
    return first                        # first is a prefix of all the others


# --- try it ---
print(longest_common_prefix(["flower", "flow", "flight"]))      # -> fl
print(repr(longest_common_prefix(["dog", "racecar", "car"])))   # -> ''
print(longest_common_prefix(["inter", "internet", "interview"]))   # -> inter
print(longest_common_prefix(["flower", "flow"]))                # -> flow
```

87. Text Justification

Greedy Line Packing, Then Spread the Spaces · Strings · Tier 3 · python/087_text_justification.py

Pack words greedily into lines of exactly max_width characters, spreading the spaces evenly with the extras in the leftmost gaps; the last line and one-word lines are left-justified: ["What", "must", "be", "acknowledgment"], 16 → ["What must be", "acknowledgment"].

```python
# --- algorithm ---
def full_justify(words, max_width):
    """Take words while they fit, then justify the line.
    O(n + L * max_width) for n words and L lines."""
    lines = []
    i = 0
    while i < len(words):
        j = i + 1                       # the line is words[i..j-1]
        width = len(words[i])           # its letters plus one space per gap
        while j < len(words) and width + 1 + len(words[j]) <= max_width:
            width = width + 1 + len(words[j])
            j += 1
        is_last = (j == len(words))
        lines.append(justify_line(words, i, j, max_width, is_last))
        i = j
    return lines


def justify_line(words, i, j, max_width, is_last):
    """Lay words[i..j-1] into exactly max_width characters. O(max_width)."""
    if is_last or j - i == 1:           # left-justify, pad on the right
        line = " ".join(words[i:j])
        return line + " " * (max_width - len(line))
    letters = 0
    for k in range(i, j):
        letters += len(words[k])
    gaps = j - i - 1
    spaces = max_width - letters        # blanks to place in the gaps
    each = spaces // gaps
    extra = spaces % gaps               # leftmost gaps get one more
    pieces = [words[i]]
    for k in range(i + 1, j):
        gap = each
        if k - i <= extra:              # gap number k - i, from 1
            gap += 1
        pieces.append(" " * gap)
        pieces.append(words[k])
    return "".join(pieces)


# --- try it ---
words = ["This", "is", "an", "example", "of", "text", "justification."]
lines = full_justify(words, 16)
print("|" + lines[0] + "|")             # -> |This    is    an|
print("|" + lines[1] + "|")             # -> |example  of text|
print("|" + lines[2] + "|")             # -> |justification.  |
lines = full_justify(["What", "must", "be", "acknowledgment", "shall"], 16)
print("|" + lines[1] + "|")             # -> |acknowledgment  |
```

88. String Compression

Run-Length Encoding with Read and Write Pointers · Strings · Tier 3 · python/088_string_compression.py

Compress a list of characters in place: each run of one repeated character becomes the character, then the run length if it is more than 1, one digit per cell; return the new length: [“a”, “a”, “b”, “b”, “c”, “c”, “c”] → 6, with [“a”, “2”, “b”, “2”, “c”, “3”].

```python
# --- algorithm ---
def compress(chars):
    """Read walks over each run, write lays down char + count digits.
    O(n) time, O(1) extra space."""
    write = 0
    read = 0
    while read < len(chars):
        ch = chars[read]
        start = read
        while read < len(chars) and chars[read] == ch:
            read += 1                   # walk to the end of the run
        chars[write] = ch
        write += 1
        count = read - start
        if count > 1:
            for digit in str(count):    # 12 is written as "1", "2"
                chars[write] = digit
                write += 1
    return write


# --- try it ---
chars = ["a", "a", "b", "b", "c", "c", "c"]
length = compress(chars)
print(length, chars[0:length])          # -> 6 ['a', '2', 'b', '2', 'c', '3']
chars = list("abbbbbbbbbbbb")
length = compress(chars)
print(length, chars[0:length])          # -> 4 ['a', 'b', '1', '2']
print(compress(["a"]), compress(list("abc")))   # -> 1 3
```

89. Reverse Words in a String

Reverse All, Then Reverse Each Word · Strings · Tier 3 · python/089_reverse_words.py

Reverse the order of the words in a string. Words are separated by one or more spaces; the result uses single spaces and has none at either end: ” the sky is blue ” → “blue is sky the”.

```python
# --- algorithm ---
def reverse_words(s):
    """Squeeze the spaces, reverse the whole array, then reverse each word.
    O(n) time; O(1) extra space on a mutable character array."""
    chars = list(s)                     # Python strings are immutable
    length = squeeze_spaces(chars)      # the words fill chars[0..length-1]
    reverse(chars, 0, length - 1)       # order flipped, letters backwards
    start = 0
    for i in range(length + 1):
        if i == length or chars[i] == " ":  # a word ends just before i
            reverse(chars, start, i - 1)    # spell it forwards again
            start = i + 1
    return "".join(chars[0:length])


def squeeze_spaces(chars):
    """Copy the words to the front, one space between them. O(n)."""
    write = 0
    for read in range(len(chars)):
        if chars[read] != " ":
            chars[write] = chars[read]
            write += 1
        elif write > 0 and chars[write - 1] != " ":
            chars[write] = " "          # keep one space after a word
            write += 1
    if write > 0 and chars[write - 1] == " ":
        write -= 1                      # drop the trailing space
    return write


def reverse(chars, i, j):
    """Reverse chars[i..j] in place, swapping the ends inward. O(j - i)."""
    while i < j:
        chars[i], chars[j] = chars[j], chars[i]
        i += 1
        j -= 1


# --- try it ---
print(reverse_words("  the sky   is blue "))   # -> blue is sky the
print(reverse_words("a good   example"))       # -> example good a
print(reverse_words("hello"))                  # -> hello
print(repr(reverse_words("    ")))             # -> ''
```

90. Multiply Strings

Grade-School Multiplication into a Digit Array · Strings · Tier 3 · python/090_multiply_strings.py

Multiply two non-negative integers given as decimal strings (no leading zeros) and return the product as a string, without converting the whole inputs to numbers: “123”, “456” → “56088”.

```python
# --- algorithm ---
def multiply(num1, num2):
    """Each digit pair adds its product into slot i + j + 1 of an m + n
    digit array, carrying left. O(m * n) time, O(m + n) space."""
    if num1 == "0" or num2 == "0":
        return "0"
    m = len(num1)
    n = len(num2)
    digits = [0] * (m + n)              # the product has at most m + n digits
    for i in range(m - 1, -1, -1):
        for j in range(n - 1, -1, -1):
            product = int(num1[i]) * int(num2[j])   # one digit times one digit
            total = digits[i + j + 1] + product
            digits[i + j + 1] = total % 10          # keep the ones digit
            digits[i + j] += total // 10            # carry one slot left
    start = 0
    if digits[0] == 0:                  # only m + n - 1 digits were needed
        start = 1
    result = []
    for k in range(start, m + n):
        result.append(str(digits[k]))
    return "".join(result)


# --- try it ---
print(multiply("123", "456"))           # -> 56088
print(multiply("2", "3"))               # -> 6
print(multiply("99", "99"))             # -> 9801
print(multiply("12345", "0"))           # -> 0
```

91. GCD and LCM

Euclid's Algorithm · Math · Tier 3 · python/091_gcd_lcm.py

The greatest common divisor of two integers is the largest number dividing both; the least common multiple is the smallest positive number both divide: gcd(48, 18) → 6, lcm(48, 18) → 144.

```python
# --- algorithm ---
def gcd(a, b):
    """Euclid: (a, b) becomes (b, a mod b) until b is 0. O(log min(a, b))."""
    a = abs(a)
    b = abs(b)
    while b != 0:
        remainder = a % b               # every common divisor survives the mod
        a = b
        b = remainder
    return a                            # gcd(a, 0) = a


def lcm(a, b):
    """Divide by the gcd before multiplying. O(log min(a, b))."""
    if a == 0 or b == 0:
        return 0
    return abs(a) // gcd(a, b) * abs(b)  # exact division first


def gcd_list(nums):
    """Fold gcd over the list from 0, since gcd(0, x) = x. O(n log max)."""
    result = 0
    for x in nums:
        result = gcd(result, x)
        if result == 1:
            break                       # it can never go below 1
    return result


# --- try it ---
print(gcd(48, 18))                      # -> 6
print(lcm(48, 18))                      # -> 144
print(gcd(7, 0))                        # -> 7
print(gcd_list([12, 18, 30]))           # -> 6
```

92. Sieve of Eratosthenes

Cross Out Multiples Starting at p squared · Math · Tier 3 · python/092_sieve_of_eratosthenes.py

List every prime below n, and test a single number for primality: n = 30 → [2, 3, 5, 7, 11, 13, 17, 19, 23, 29]; 91 → not prime (7 * 13).

```python
# --- algorithm ---
def sieve(n):
    """Primes below n: each surviving p crosses out p*p, p*p + p, ...
    O(n log log n) time, O(n) space."""
    crossed = [False] * n           # True once a smaller prime divides i
    p = 2
    while p * p < n:
        if not crossed[p]:          # p is prime: nothing crossed it out
            multiple = p * p        # below p * p: already crossed
            while multiple < n:
                crossed[multiple] = True
                multiple += p
        p += 1
    primes = []
    for i in range(2, n):
        if not crossed[i]:
            primes.append(i)
    return primes


def is_prime(n):
    """Trial division by 2, 3, 4, ... while d * d <= n. O(sqrt(n))."""
    if n < 2:
        return False
    d = 2
    while d * d <= n:               # d and n / d: one is at most sqrt(n)
        if n % d == 0:
            return False
        d += 1
    return True


# --- try it ---
print(sieve(30))                # -> [2, 3, 5, 7, 11, 13, 17, 19, 23, 29]
print(len(sieve(10)))           # -> 4
print(is_prime(91))             # -> False
print(is_prime(97))             # -> True
```

93. Fast Exponentiation

Square and Multiply · Math · Tier 3 · python/093_fast_exponentiation.py

Compute x to the power n with O(log n) multiplications, for a negative n as well, and the same power modulo m: 3 ^ 13 → 1594323, 2 ^ -2 → 0.25.

```python
# --- algorithm ---
def power(x, n):
    """Square x once per bit of n; multiply it in where the bit is 1.
    O(log |n|) multiplications."""
    if n < 0:
        x = 1 / x                       # x^-n = (1 / x)^n
        n = -n
    result = 1
    while n > 0:
        if n % 2 == 1:                  # lowest bit is 1: this square counts
            result = result * x
        x = x * x                       # x, x^2, x^4, x^8, ...
        n = n // 2                      # drop the lowest bit
    return result


def mod_power(x, n, m):
    """The same loop with every product reduced mod m. O(log n)."""
    result = 1 % m                      # 0 when m is 1
    x = x % m
    while n > 0:
        if n % 2 == 1:
            result = result * x % m
        x = x * x % m                   # numbers stay below m * m
        n = n // 2
    return result


# --- try it ---
print(power(3, 13))                     # -> 1594323
print(power(2, -2))                     # -> 0.25
print(power(7, 0))                      # -> 1
print(mod_power(3, 13, 1000))           # -> 323
print(mod_power(2, 100, 1000000007))    # -> 976371285
```

94. Reservoir Sampling

Keep the i-th Item with Probability k / i · Math · Tier 3 · python/094_reservoir_sampling.py

Choose k items uniformly at random from a stream of unknown length, in one pass with O(k) memory, so that every item is kept with the same chance k / n: stream 10, 20, 30, 40, 50 with k = 2 → each item kept 2 / 5 of the time.

```python
import random                           # randint(a, b) includes both ends


# --- algorithm ---
def reservoir_sample(stream, k):
    """Keep the first k items; item i then replaces a random slot with
    probability k / i. O(n) time, O(k) space."""
    reservoir = []
    i = 0
    for item in stream:
        i += 1                          # item is the i-th seen (1-based)
        if i <= k:
            reservoir.append(item)      # fill phase
        else:
            j = random.randint(1, i)    # uniform over 1..i
            if j <= k:                  # chance k / i
                reservoir[j - 1] = item  # evict a uniform slot
    return reservoir


def is_uniform(stream, k, trials):
    """Each item should be kept about trials * k / n times. O(trials * n)."""
    counts = {}
    for t in range(trials):
        sample = reservoir_sample(stream, k)
        for item in sample:
            counts[item] = counts.get(item, 0) + 1
    expected = trials * k / len(stream)
    for item in stream:
        if abs(counts.get(item, 0) - expected) > 0.1 * expected:
            return False                # more than 10 percent off
    return True


# --- try it ---
random.seed(7)
print(len(reservoir_sample(range(1000), 10)))           # -> 10
print(reservoir_sample([7, 8, 9], 5))                   # -> [7, 8, 9]
print(is_uniform([10, 20, 30, 40, 50], 2, 20000))       # -> True
print(is_uniform([1, 2, 3, 4, 5, 6, 7, 8], 1, 20000))   # -> True
```

95. Orientation and Segment Intersection

The Cross Product Decides Left or Right · Geometry · Tier 3 · python/095_orientation_segment_intersection.py

Decide whether the path o → a → b turns left, turns right or runs straight, and whether two segments share a point. Points are (x, y) integer pairs: segments (0, 0)-(4, 4) and (0, 4)-(4, 0) → True, they cross at (2, 2).

```python
# --- algorithm ---
def cross(o, a, b):
    """Cross product of o->a and o->b: > 0 left turn, < 0 right turn,
    0 straight. O(1)."""
    return (a[0] - o[0]) * (b[1] - o[1]) - (a[1] - o[1]) * (b[0] - o[0])


def on_segment(p, q, r):
    """For r on the line pq: is r inside the box spanned by p and q? O(1)."""
    inside_x = min(p[0], q[0]) <= r[0] <= max(p[0], q[0])
    inside_y = min(p[1], q[1]) <= r[1] <= max(p[1], q[1])
    return inside_x and inside_y


def segments_intersect(p1, p2, q1, q2):
    """Each segment's ends lie on opposite sides of the other's line, or an
    end touches the other segment. O(1)."""
    d1 = cross(q1, q2, p1)              # side of p1 seen from q1 -> q2
    d2 = cross(q1, q2, p2)
    d3 = cross(p1, p2, q1)              # side of q1 seen from p1 -> p2
    d4 = cross(p1, p2, q2)
    if d1 * d2 < 0 and d3 * d4 < 0:
        return True                     # both pairs straddle: a clean crossing
    if d1 == 0 and on_segment(q1, q2, p1):
        return True                     # p1 lies on segment q1 q2
    if d2 == 0 and on_segment(q1, q2, p2):
        return True
    if d3 == 0 and on_segment(p1, p2, q1):
        return True
    if d4 == 0 and on_segment(p1, p2, q2):
        return True
    return False


# --- try it ---
print(cross((0, 0), (4, 0), (2, 3)))                        # -> 12
print(segments_intersect((0, 0), (4, 4), (0, 4), (4, 0)))   # -> True
print(segments_intersect((0, 0), (2, 2), (3, 3), (5, 5)))   # -> False
print(segments_intersect((0, 0), (4, 0), (2, 0), (2, 3)))   # -> True
print(segments_intersect((0, 0), (4, 0), (0, 1), (4, 2)))   # -> False
```

96. Convex Hull

Andrew's Monotone Chain · Geometry · Tier 3 · python/096_convex_hull.py

Return the corners of the smallest convex polygon containing every (x, y) point, counter-clockwise from the leftmost one (the lowest on a tie): (0, 0) (1, 3) (2, 1) (2, 2) (3, 2) (4, 0) (4, 4) → (0, 0) (4, 0) (4, 4) (1, 3)

```python
# --- algorithm ---
def cross(o, a, b):
    """> 0 when o -> a -> b turns left (counter-clockwise), as in #95. O(1)."""
    return (a[0] - o[0]) * (b[1] - o[1]) - (a[1] - o[1]) * (b[0] - o[0])


def half_hull(pts):
    """One chain: a stack that keeps only left turns. O(n), since every
    point is pushed once and popped at most once."""
    chain = []                          # used as a stack: chain[-1] is the top
    for p in pts:
        while len(chain) >= 2 and cross(chain[-2], chain[-1], p) <= 0:
            chain.pop()                 # last turn is not counter-clockwise
        chain.append(p)
    return chain


def convex_hull(points):
    """Sort by (x, y); the lower chain runs left to right, the upper chain
    right to left. O(n log n) for the sort."""
    pts = sorted(set(points))           # by x, then y; duplicates dropped
    if len(pts) <= 2:
        return pts
    lower = half_hull(pts)
    upper = half_hull(pts[::-1])
    return lower[:-1] + upper[:-1]      # each chain ends where the other starts


# --- try it ---
pts = [(0, 0), (1, 3), (2, 1), (2, 2), (3, 2), (4, 0), (4, 4)]
print(convex_hull(pts))                 # -> [(0, 0), (4, 0), (4, 4), (1, 3)]
print(convex_hull([(0, 0), (1, 1), (2, 2), (1, 1)]))    # -> [(0, 0), (2, 2)]
print(convex_hull([(5, 5)]))            # -> [(5, 5)]
square = [(0, 0), (2, 0), (2, 2), (0, 2), (1, 1), (1, 0)]
print(convex_hull(square))              # -> [(0, 0), (2, 0), (2, 2), (0, 2)]
```

97. Max Points on a Line

Slopes as Reduced Fractions · Geometry · Tier 3 · python/097_max_points_on_a_line.py

Given distinct points with integer coordinates, return the largest number of them that lie on one straight line: (1, 1) (3, 2) (5, 3) (4, 1) (2, 3) (1, 4) → 4, the line x + y = 5.

```python
import math                             # gcd is Euclid's algorithm, #91


# --- algorithm ---
def slope_key(p, q):
    """Direction p -> q as a reduced fraction (dy, dx) with dx > 0;
    every vertical pair gets (1, 0). O(log C) for the gcd."""
    dx = q[0] - p[0]
    dy = q[1] - p[1]
    if dx == 0:
        return (1, 0)                   # vertical line
    if dx < 0:
        dx = -dx                        # p -> q and q -> p: the same line
        dy = -dy
    g = math.gcd(dy, dx)                # positive, since dx > 0
    return (dy // g, dx // g)           # exact divisions


def max_points(points):
    """Anchor each point and group the later points by slope; the biggest
    group plus the anchor is a line. O(n^2) slopes."""
    n = len(points)
    if n <= 2:
        return n
    best = 0
    for i in range(n):
        counts = {}                     # slope key -> points on that line
        for j in range(i + 1, n):
            key = slope_key(points[i], points[j])
            counts[key] = counts.get(key, 0) + 1
            if counts[key] + 1 > best:
                best = counts[key] + 1  # + 1 for the anchor itself
    return best


# --- try it ---
pts = [(1, 1), (3, 2), (5, 3), (4, 1), (2, 3), (1, 4)]
print(max_points(pts))                          # -> 4
print(max_points([(1, 1), (2, 2), (3, 3)]))     # -> 3
print(max_points([(0, 0), (0, 5), (0, -2), (3, 1)]))  # -> 3
print(slope_key((3, 2), (1, 4)))                # -> (-1, 1)
```

98. Count Set Bits

Brian Kernighan's n & (n - 1) · Bit Manipulation · Tier 3 · python/098_count_set_bits.py

Count the 1 bits in the binary form of a non-negative integer, test for a power of two, and count the 1 bits of every number from 0 to n: $44 = 101100 \to 3$ set bits.

```python
# --- algorithm ---
def count_set_bits(n):
    """Kernighan: n & (n - 1) clears the lowest set bit. O(set bits)."""
    count = 0
    while n > 0:
        n = n & (n - 1)                # the lowest 1 becomes 0
        count += 1
    return count


def is_power_of_two(n):
    """A power of two has exactly one set bit. O(1)."""
    return n > 0 and (n & (n - 1)) == 0


def count_bits_up_to(n):
    """bits[i] = bits[i >> 1] + (i & 1) for every i in 0..n. O(n)."""
    bits = [0] * (n + 1)
    for i in range(1, n + 1):
        bits[i] = bits[i >> 1] + (i & 1)    # i >> 1 drops i's last bit
    return bits


# --- try it ---
print(count_set_bits(44))          # -> 3
print(count_set_bits(0))           # -> 0
print(is_power_of_two(64))         # -> True
print(is_power_of_two(0))          # -> False
print(count_bits_up_to(5))         # -> [0, 1, 1, 2, 1, 2]
```

99. Bitmask Subset Enumeration

Every Integer from 0 to 2^n - 1 Is a Subset · Bit Manipulation · Tier 3 · python/099_bitmask_subsets.py

List every subset of n distinct elements by counting in binary (bit j of the mask says whether element j is in), and list every submask of a mask: [1, 2, 3] → [[], [1], [2], [1, 2], [3], [1, 3], [2, 3], [1, 2, 3]].

```python
# --- algorithm ---
def subsets_by_mask(nums):
    """Mask m picks nums[j] when bit j of m is set. O(n * 2^n)."""
    n = len(nums)
    result = []
    for mask in range(1 << n):          # 1 << n is 2^n: one mask per subset
        subset = []
        for j in range(n):
            if (mask >> j) & 1:         # bit j is set: nums[j] is in
                subset.append(nums[j])
        result.append(subset)
    return result


def submasks(mask):
    """Every submask of mask, largest first. O(2^k) for k set bits."""
    result = []
    sub = mask
    while sub > 0:
        result.append(sub)
        sub = (sub - 1) & mask          # borrow, then keep only mask's bits
    result.append(0)                    # the empty submask ends the walk
    return result


# --- try it ---
result = subsets_by_mask([1, 2, 3])
print(result)   # -> [[], [1], [2], [1, 2], [3], [1, 3], [2, 3], [1, 2, 3]]
print(subsets_by_mask([]))              # -> [[]]
print(submasks(13))                     # -> [13, 12, 9, 8, 5, 4, 1, 0]
print(submasks(0))                      # -> [0]
```

100. Rate Limiter

Token Bucket and Sliding Log · Design · Tier 3 · python/100_rate_limiter.py

Decide for each request at time t (in seconds) whether to let it through, so a client gets at most a fixed rate: with 3 requests per 6 seconds, requests at times 0, 0, 0, 0, 6 → allowed, allowed, allowed, rejected, allowed.

```python
from collections import deque      # popleft is O(1)


# --- helpers ---
def allowed_times(limiter, times):
    """Send one request at each time, in order; return the times let through."""
    allowed = []
    for t in times:
        if limiter.allow(t):
            allowed.append(t)
    return allowed


# --- algorithm ---
class TokenBucket:
    """Up to capacity tokens, refilled at rate per second; a request spends
    one. O(1) time per request and O(1) memory."""

    def __init__(self, capacity, rate):
        self.capacity = capacity
        self.rate = rate               # tokens added per second
        self.tokens = capacity         # start full: a burst may pass
        self.last = 0                  # time of the last refill

    def allow(self, t):
        """Refill for the time since the last call, then spend a token. O(1)."""
        refill = (t - self.last) * self.rate    # earned since the last call
        self.tokens = min(self.capacity, self.tokens + refill)
        self.last = t
        if self.tokens >= 1:
            self.tokens -= 1
            return True
        return False


class SlidingLog:
    """The times of the requests allowed in the last window seconds.
    O(1) amortized time per request, O(limit) memory."""

    def __init__(self, limit, window):
        self.limit = limit
        self.window = window
        self.log = deque()             # allowed times, oldest first

    def allow(self, t):
        """Drop expired times; allow if under the limit. O(1) amortized."""
        while len(self.log) > 0 and self.log[0] <= t - self.window:
            self.log.popleft()         # older than the window: expired
```

```
        if len(self.log) < self.limit:
            self.log.append(t)
            return True
        return False


# --- try it ---
times = [0, 0, 0, 0, 1, 2, 4, 6, 6, 7]
print(allowed_times(TokenBucket(3, 0.5), times))  # -> [0, 0, 0, 2, 4, 6]
print(allowed_times(SlidingLog(3, 6), times))     # -> [0, 0, 0, 6, 6, 7]
print(allowed_times(SlidingLog(1, 10), [1, 3, 10, 11]))   # -> [1, 11]
```